CSE 385: Analysis of Algorithms (Honors)

Lecture 14
(Backtracking)

Rezaul A. Chowdhury
Department of Computer Science
SUNY Stony Brook
Spring 2026

Backtracking

Backtracking is an algorithm design technique in which one tries to construct a solution to a computational problem incrementally.

Backtracking algorithms are recursive.

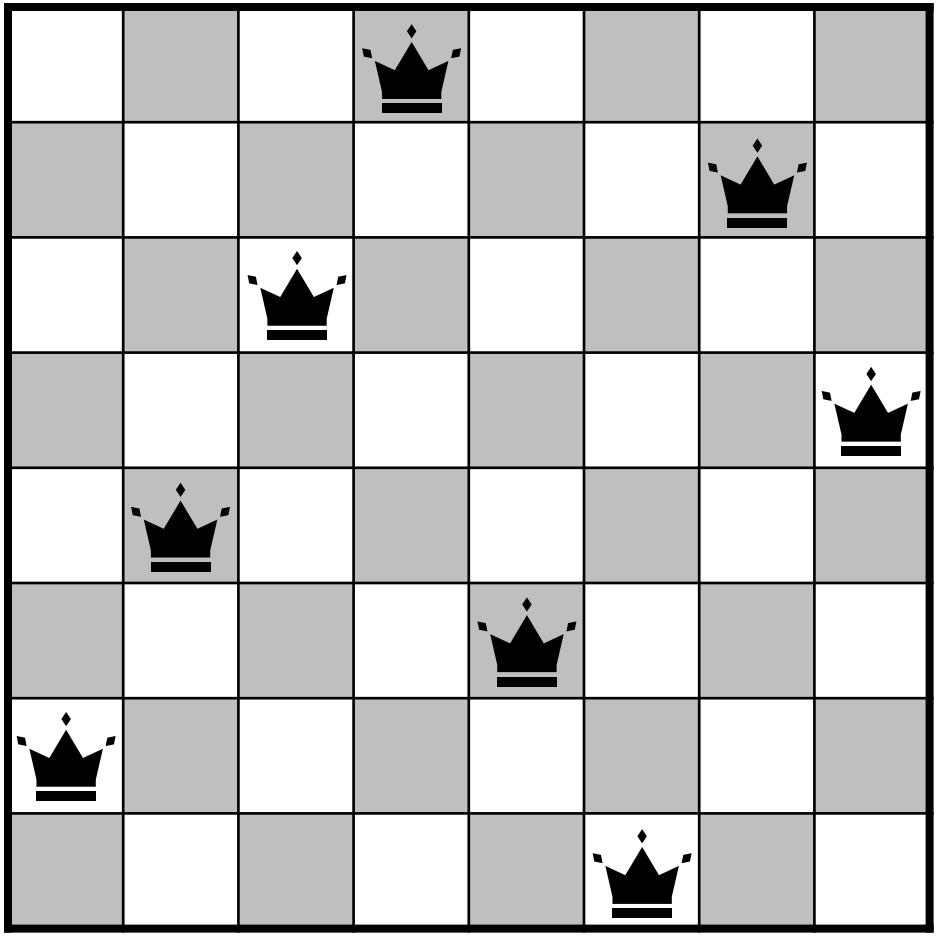
The solution is built component by component.

If at any point a component must be chosen from multiple alternatives, the algorithm tries each of them recursively, and chooses the one that produces the best results (if one exists).

If at any level of recursion it realizes that the current choice of a component is a bad one, it backtracks to the previous level of recursion.

The n-Queens Problem

Given an $n \times n$ chessboard, one must place n-queens on it so that no two queens are in attacking positions.



The n-Queens Problem

Given an $n \times n$ chessboard, one must place n-queens on it so that no two queens are in attacking positions.

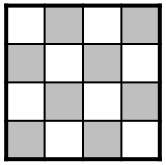
When called as *PLACE-QUEENS*($Q[1:n],\ n,\ 1$), the following algorithm returns a solution to the n queen problem in $Q[1:n]$, where for $1 \leq r \leq n$, $Q[r]$ gives the column coordinate of the queen to placed in row r.

```
PLACE-QUEENS ( Q, n, r )
1.    if r > n then
2.        return TRUE
3.    else
4.        for i ← 1 to n do
5.            safe ← TRUE
6.            for j ← 1 to r − 1 do
7.                if Q[j] = i or Q[j] = i + r − j or Q[j] = i − r + j then
8.                    safe ← FALSE
9.            if safe = TRUE then
10.               Q[r] ← i
11.               if PLACE-QUEENS ( Q, n, r + 1 ) = TRUE then
12.                   return TRUE
13.   return FALSE
```

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

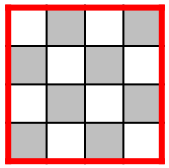
PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).



The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

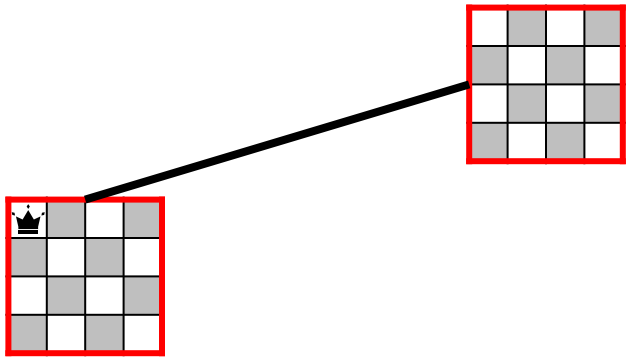
PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).



The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

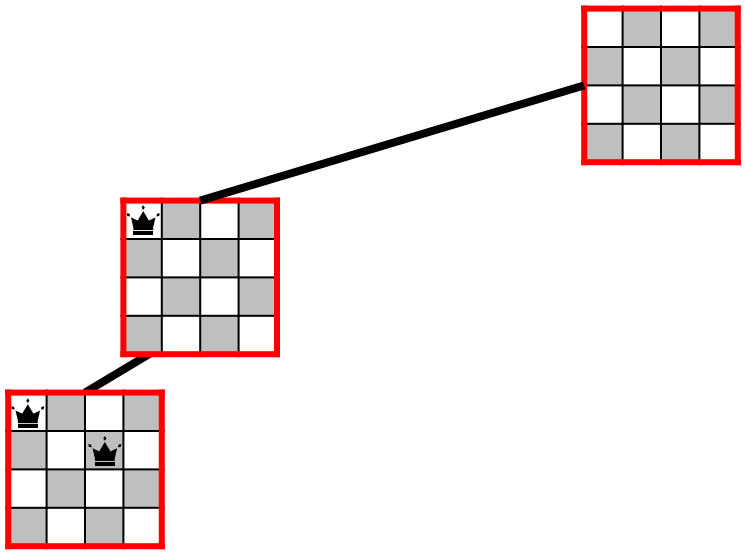


Forward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

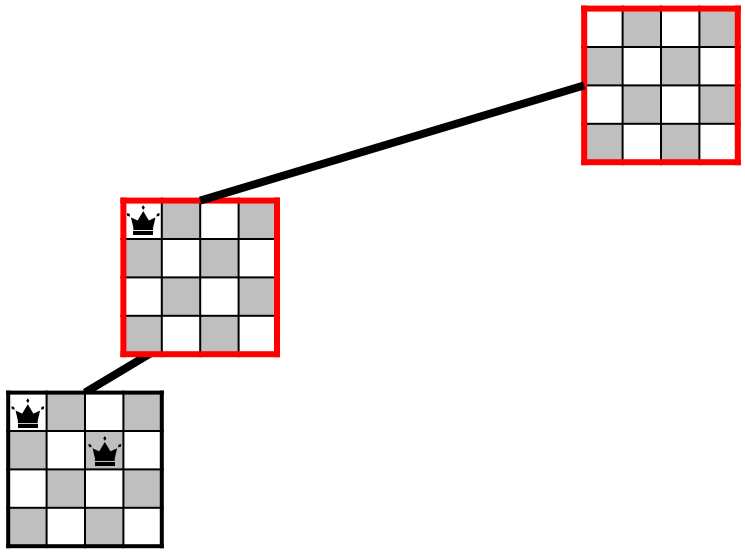


Forward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

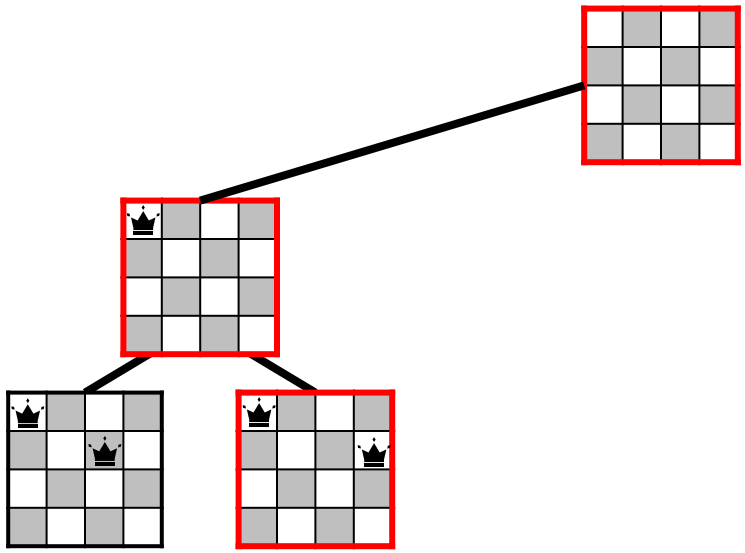


Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

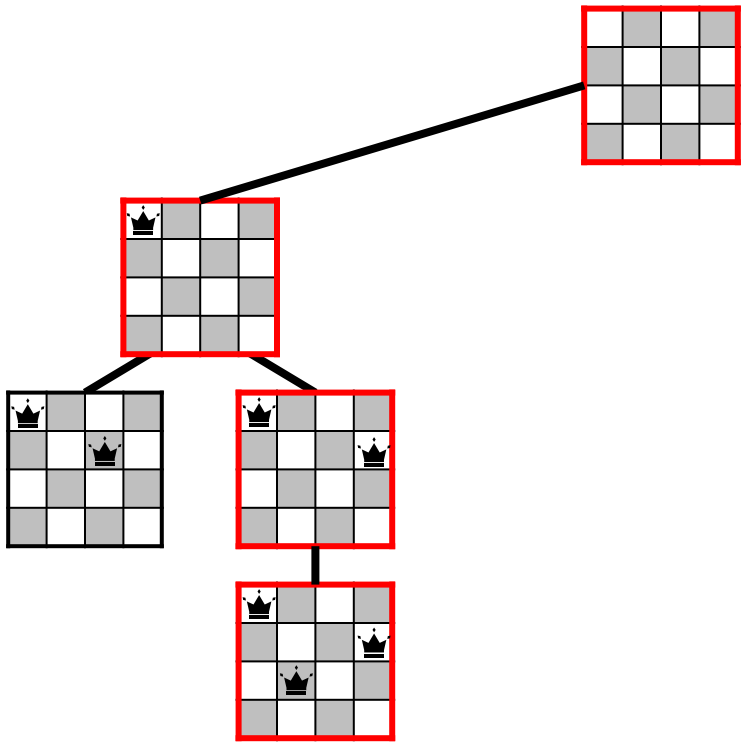


Forward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

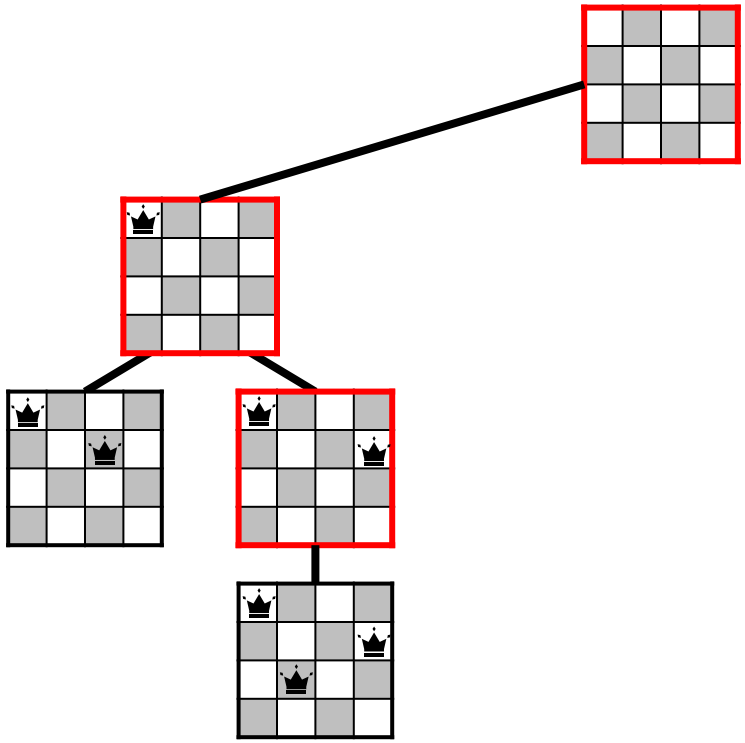


Forward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

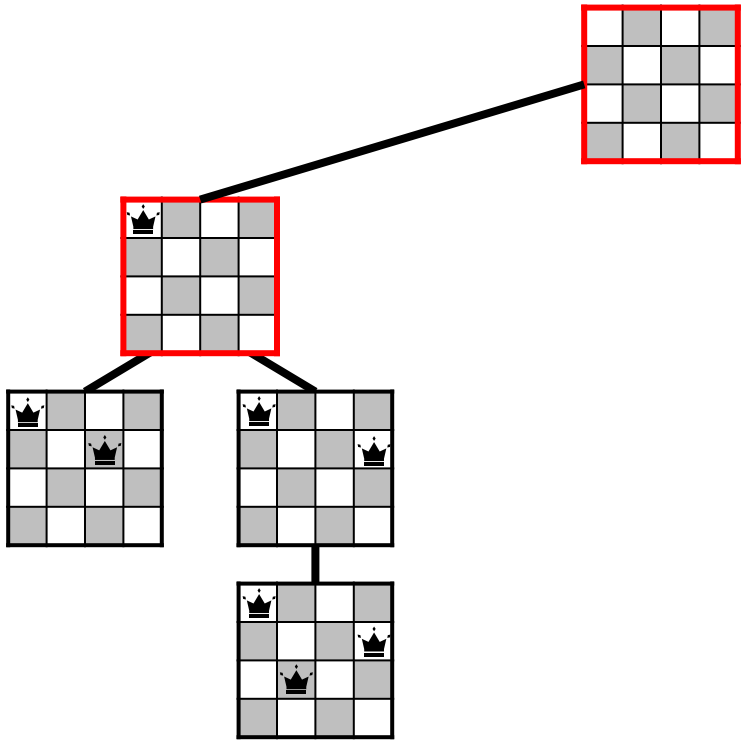


Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

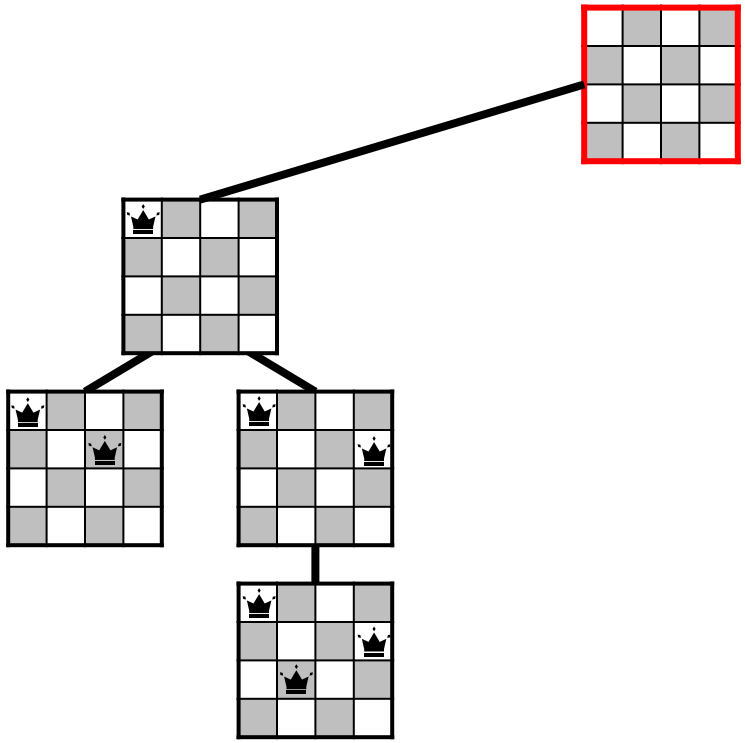


Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

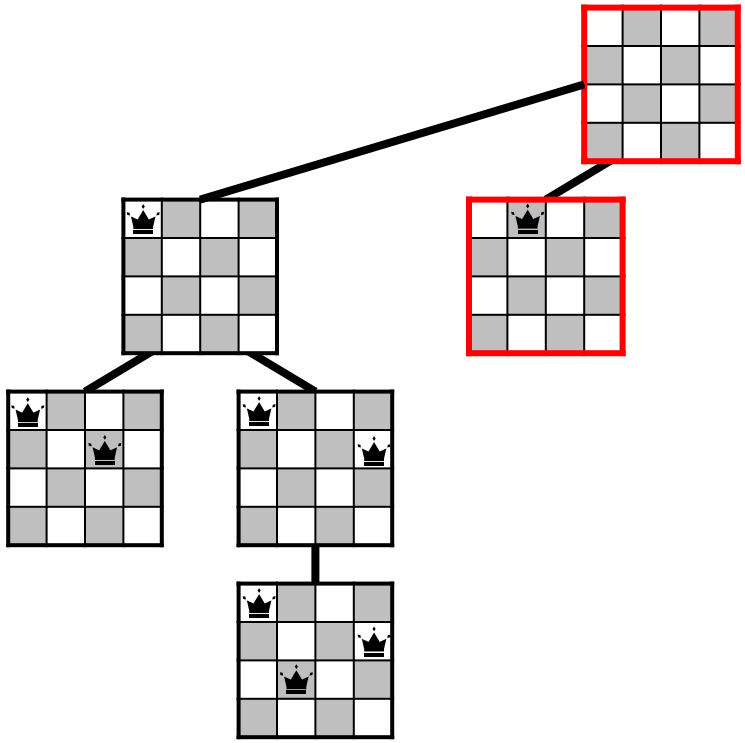


Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

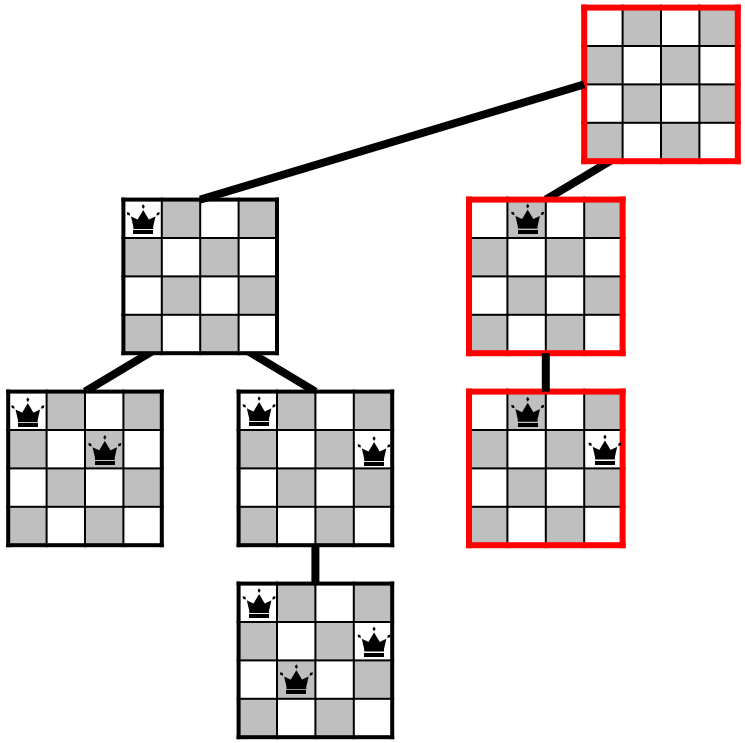


Forward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

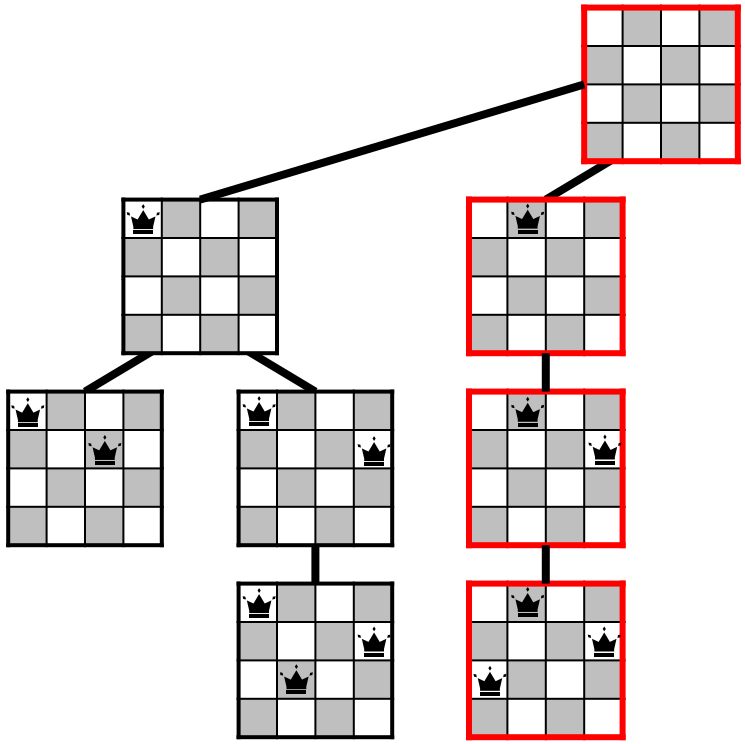


Forward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

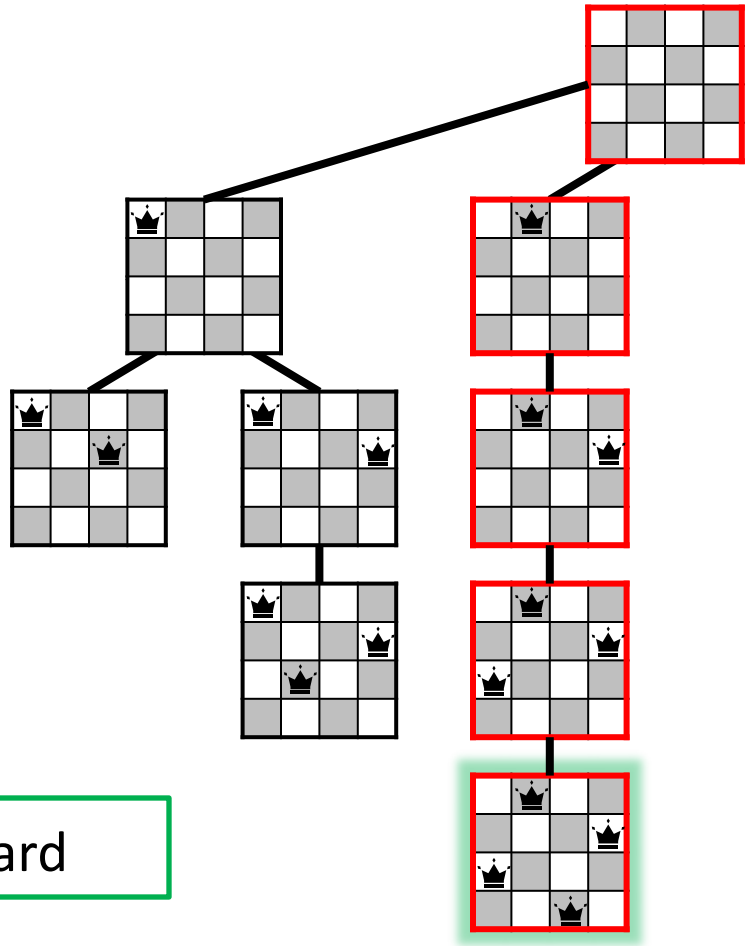


Forward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).



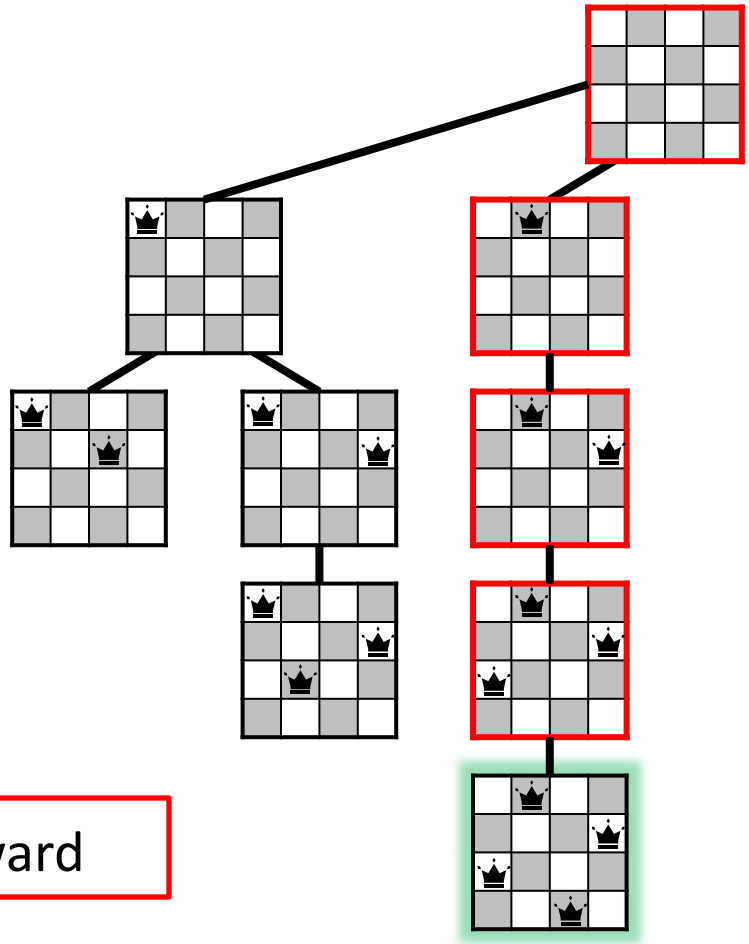
Forward

Solution Found

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

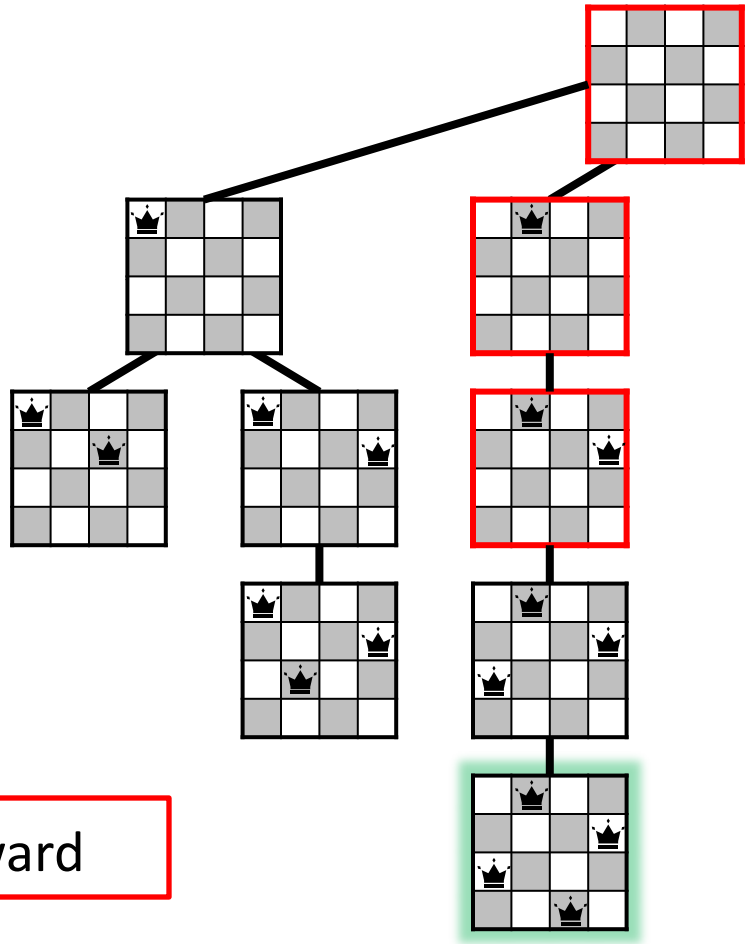


Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

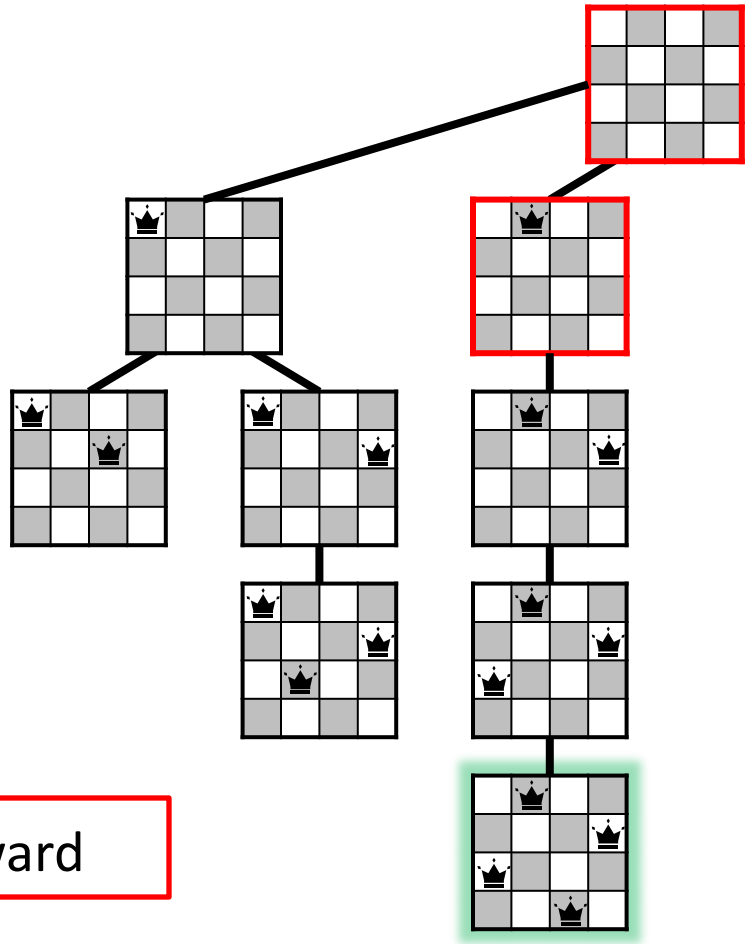


Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

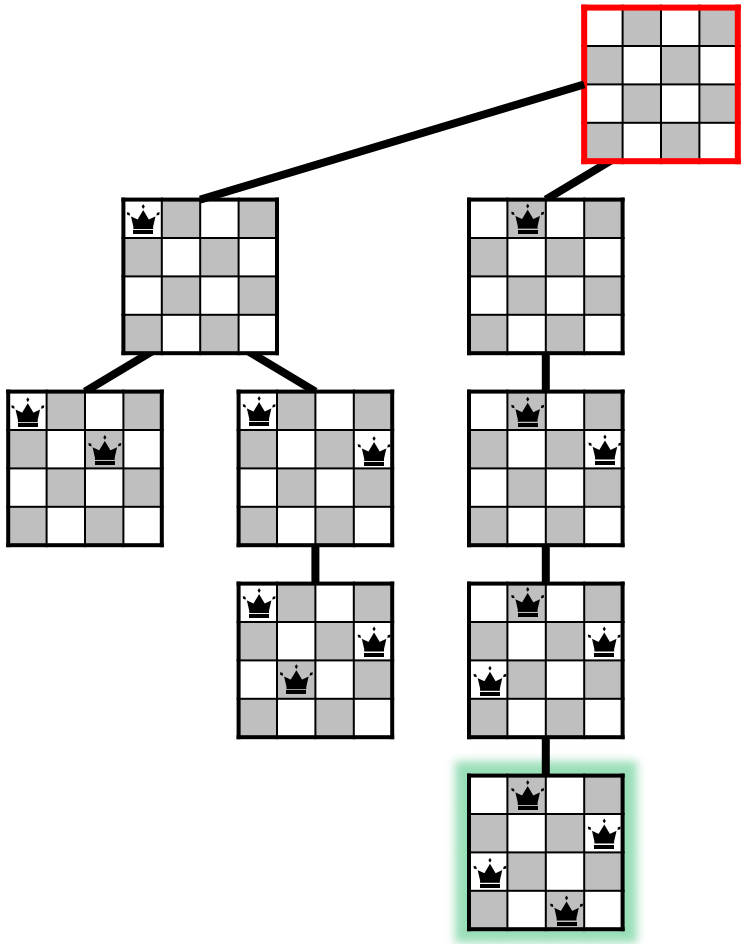


Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

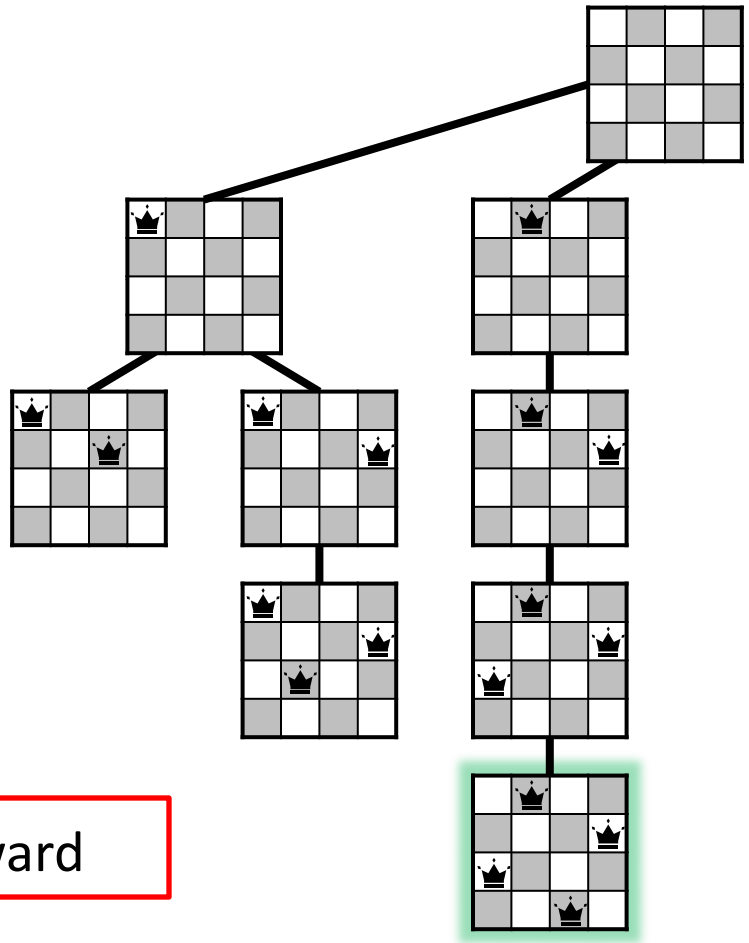
PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).



The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).



Backward

PLACE-QUEENS

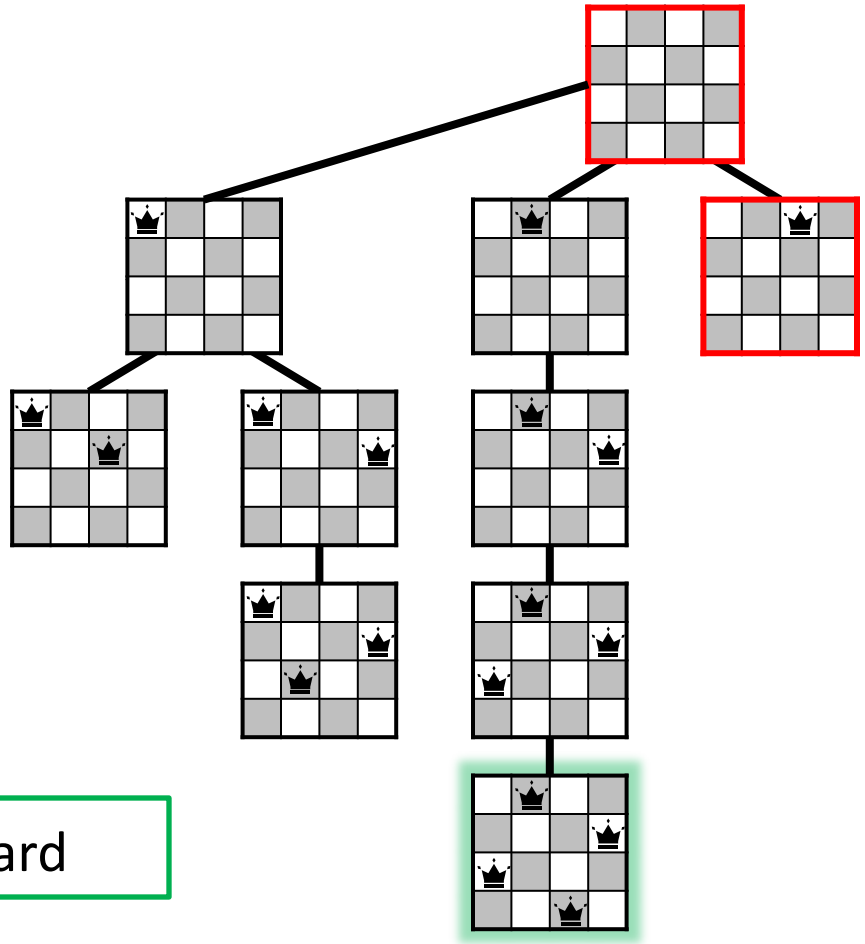
Done!

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



Forward

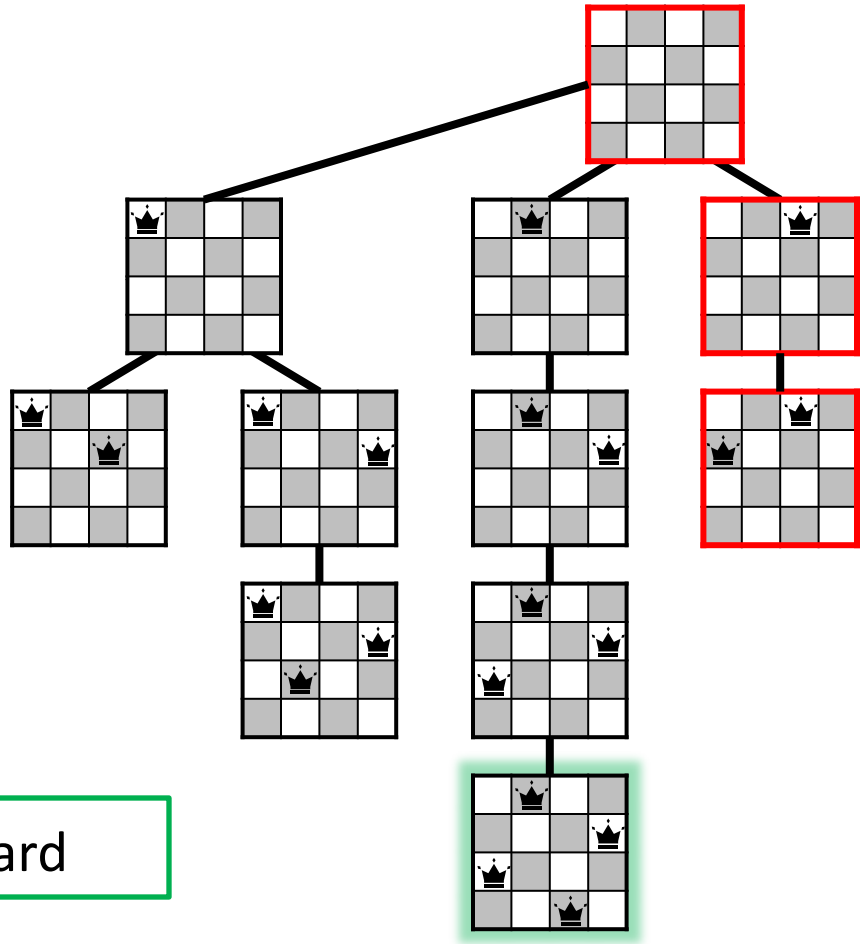
Find all Solutions!

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



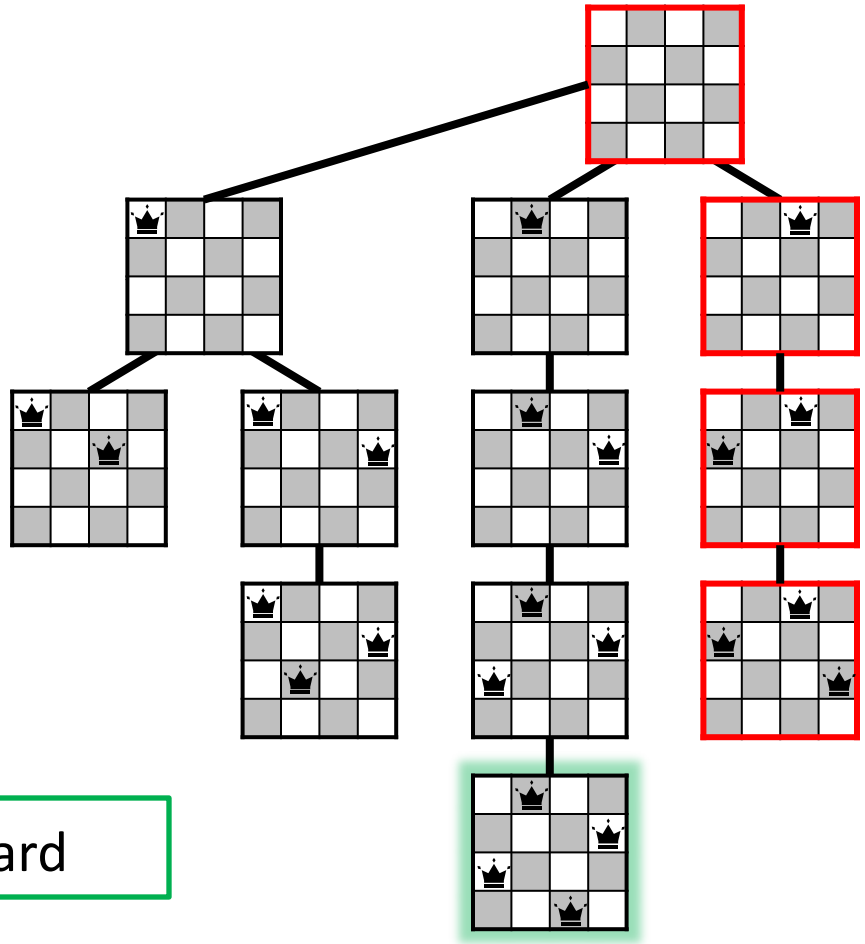
Forward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



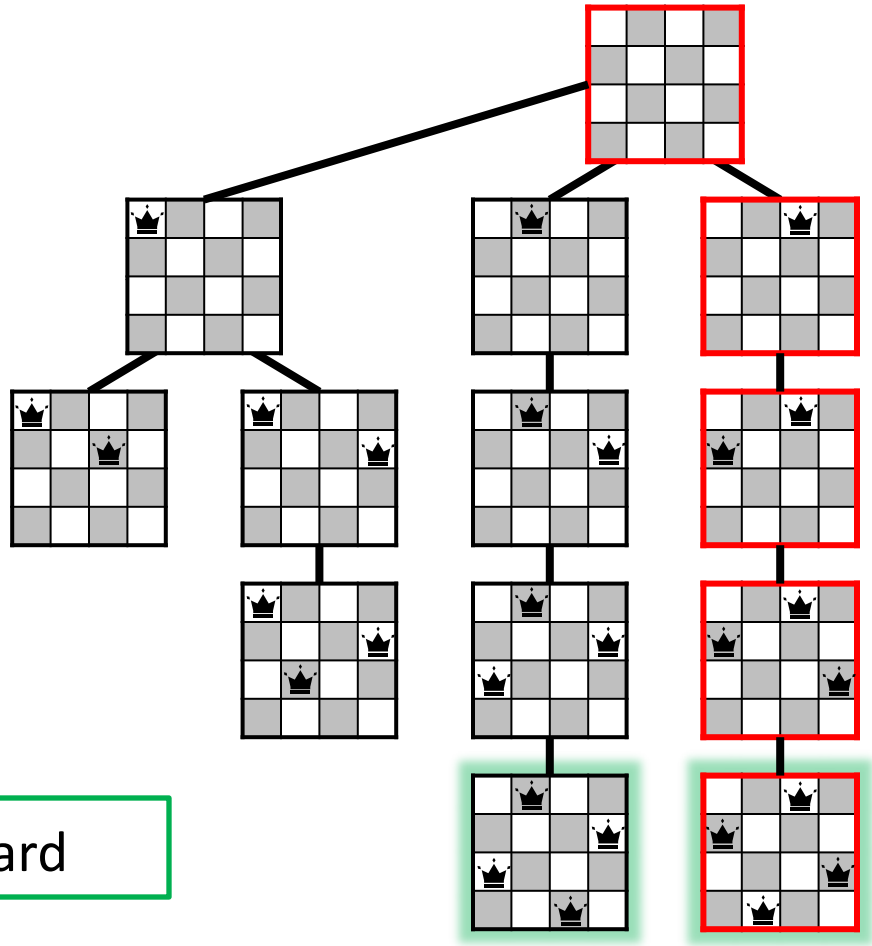
Forward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



Forward

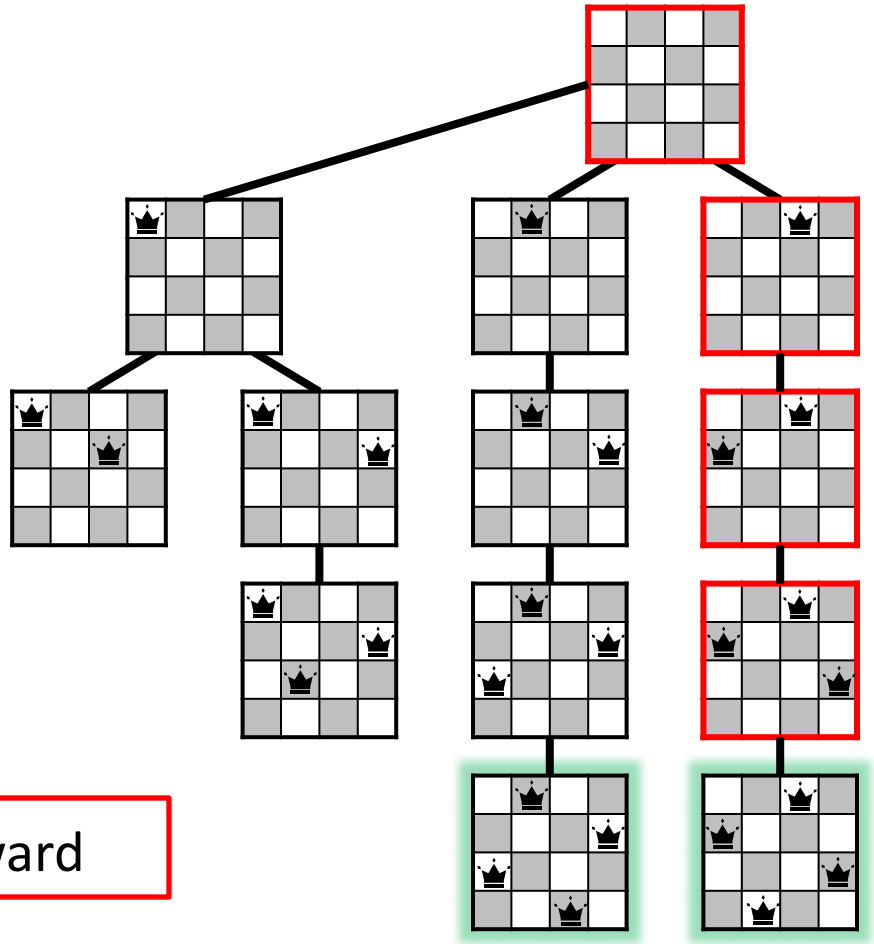
Solution Found

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



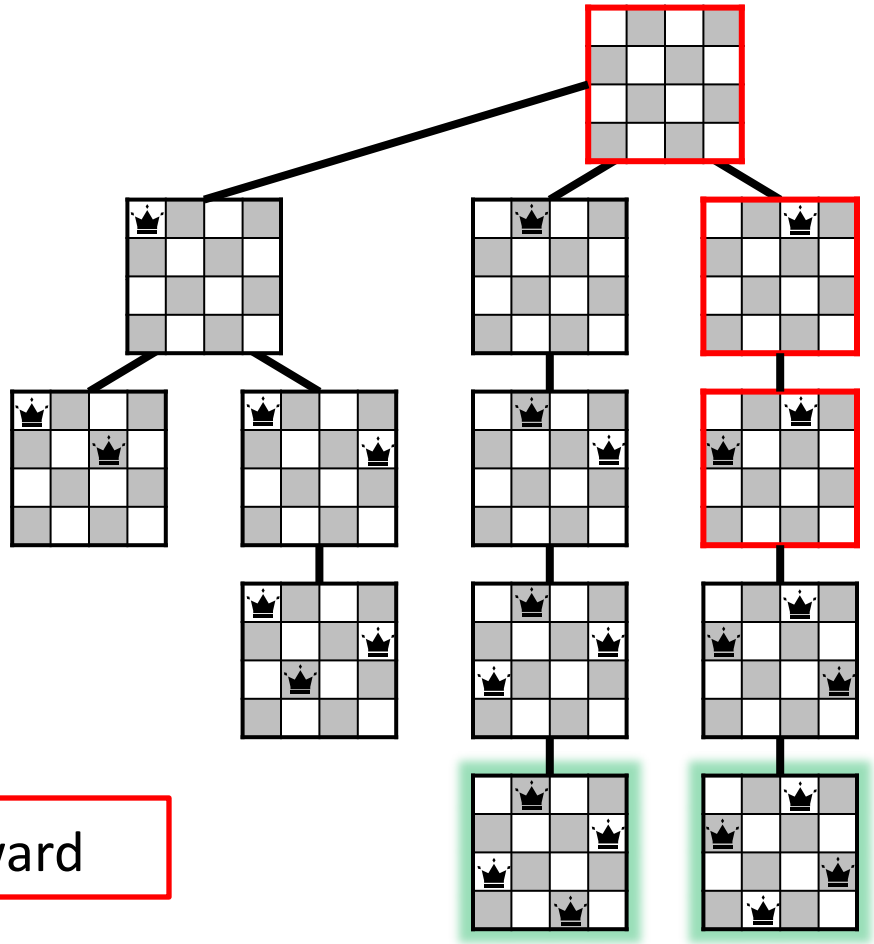
Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



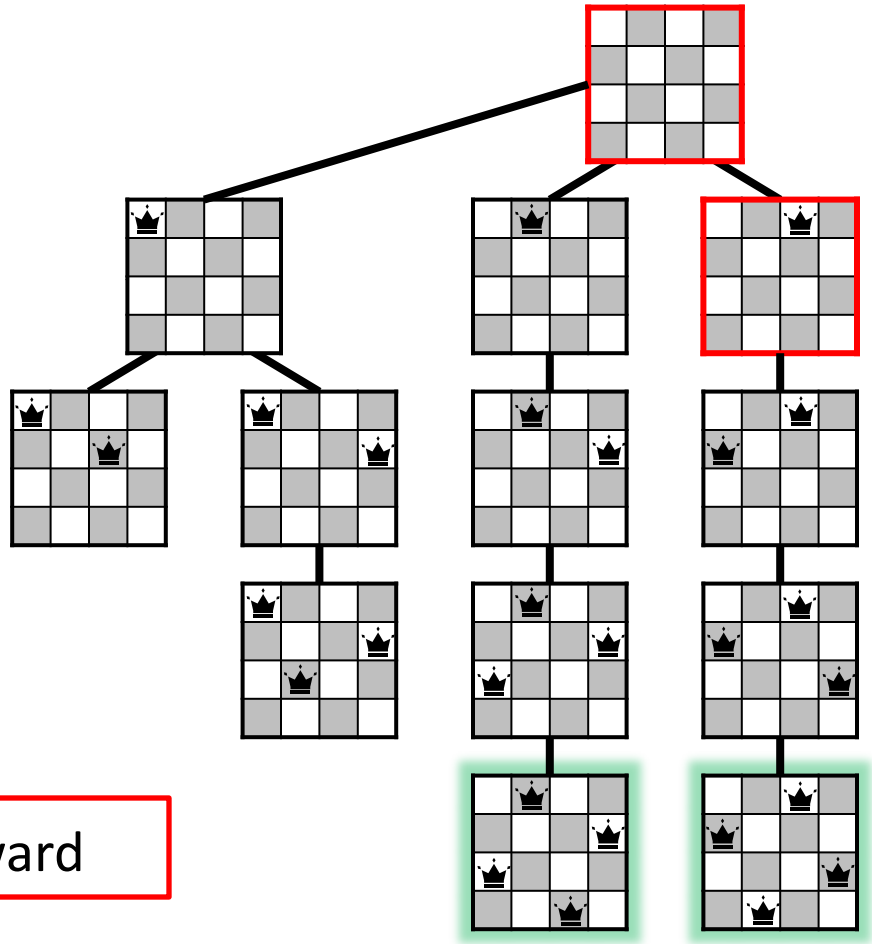
Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



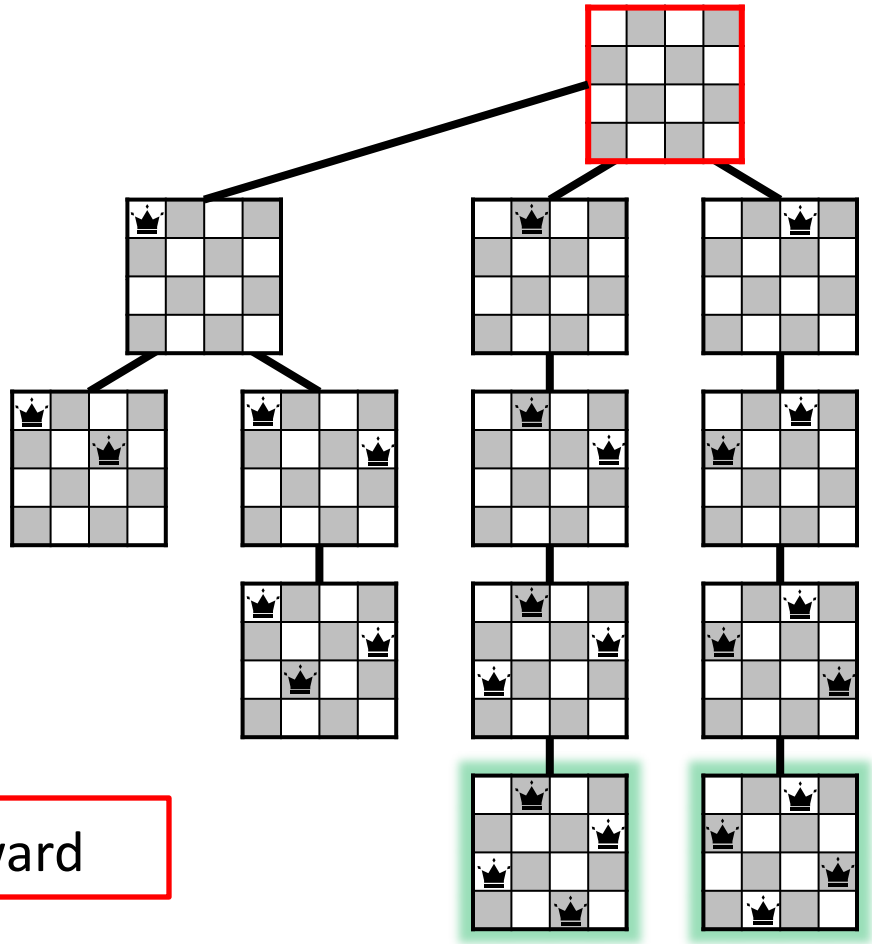
Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



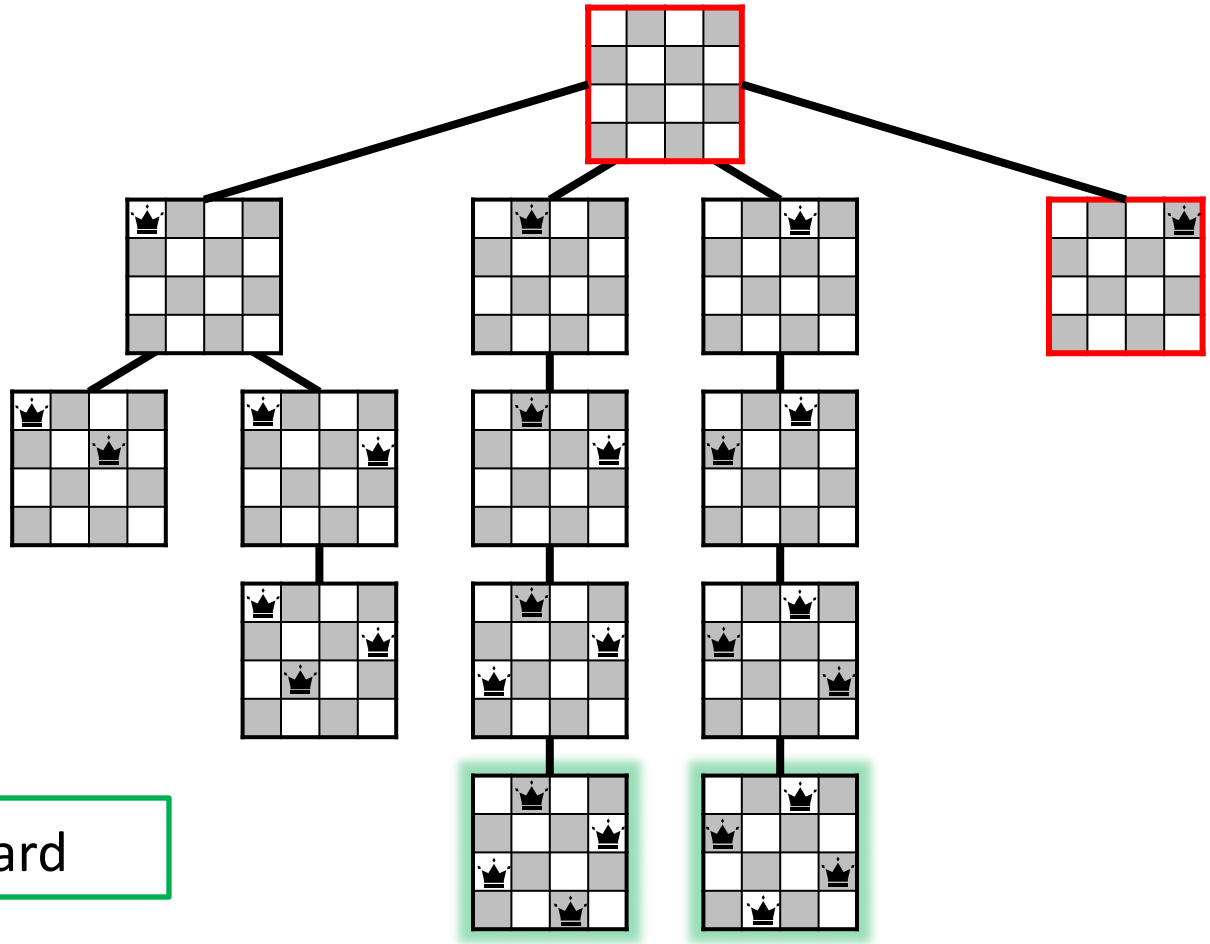
Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



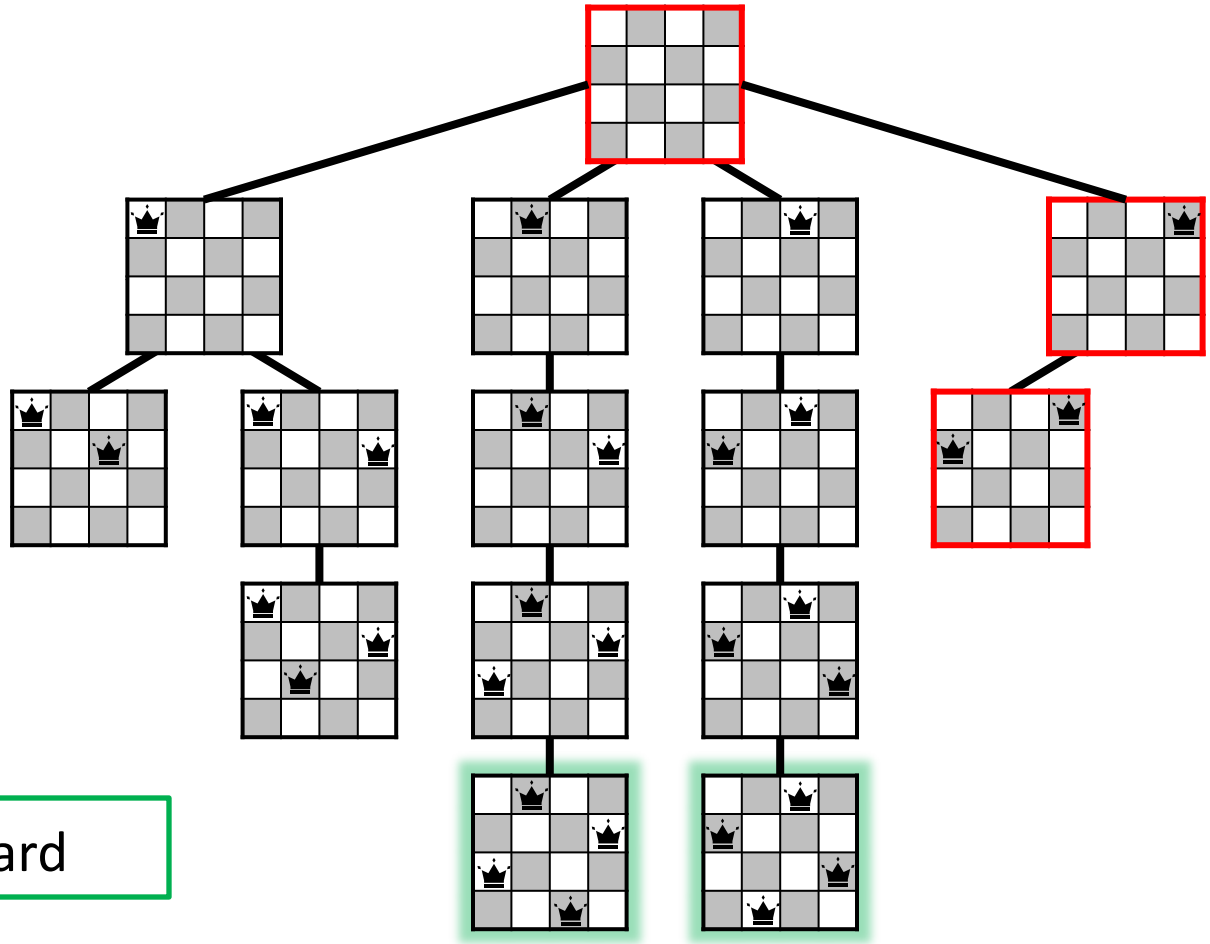
Forward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



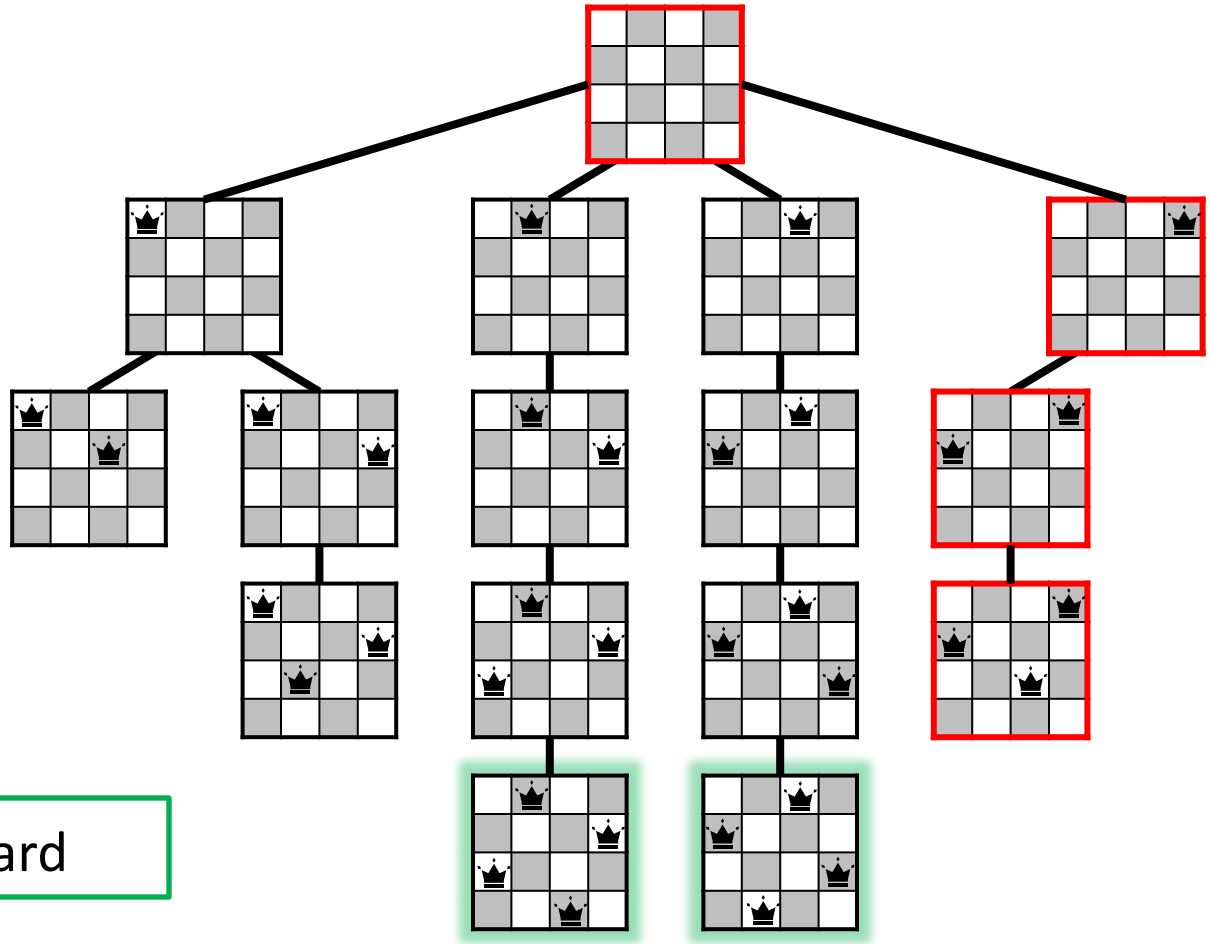
Forward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



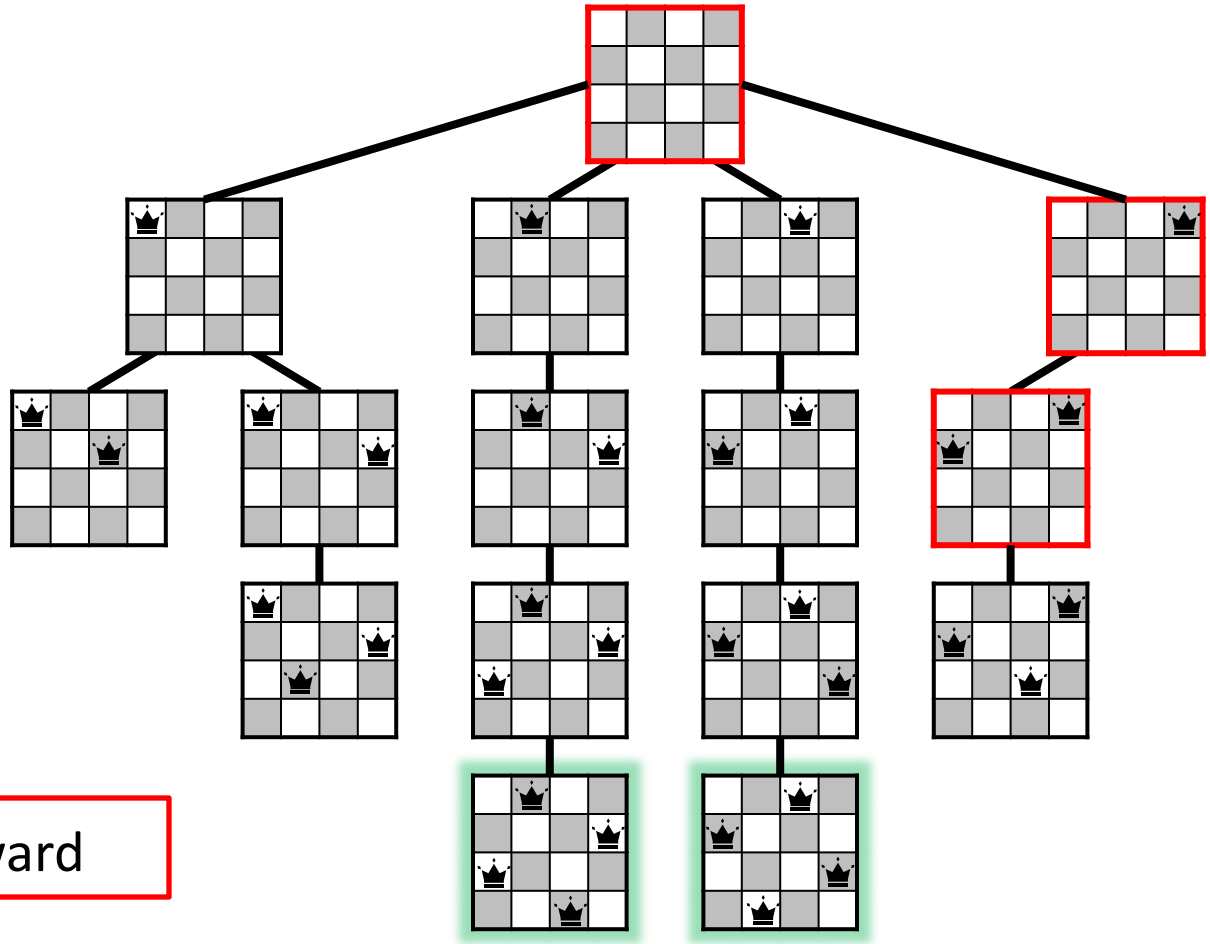
Forward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



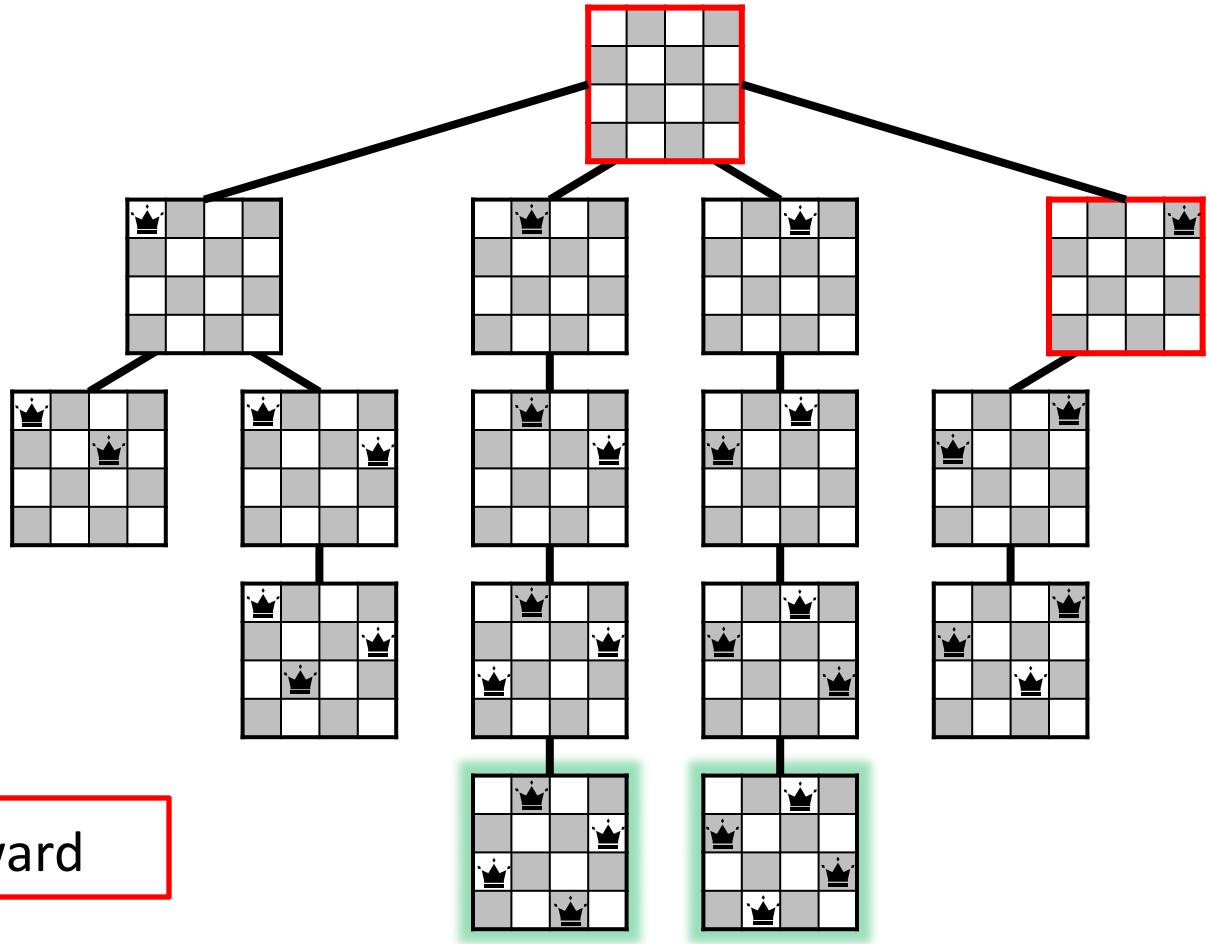
Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



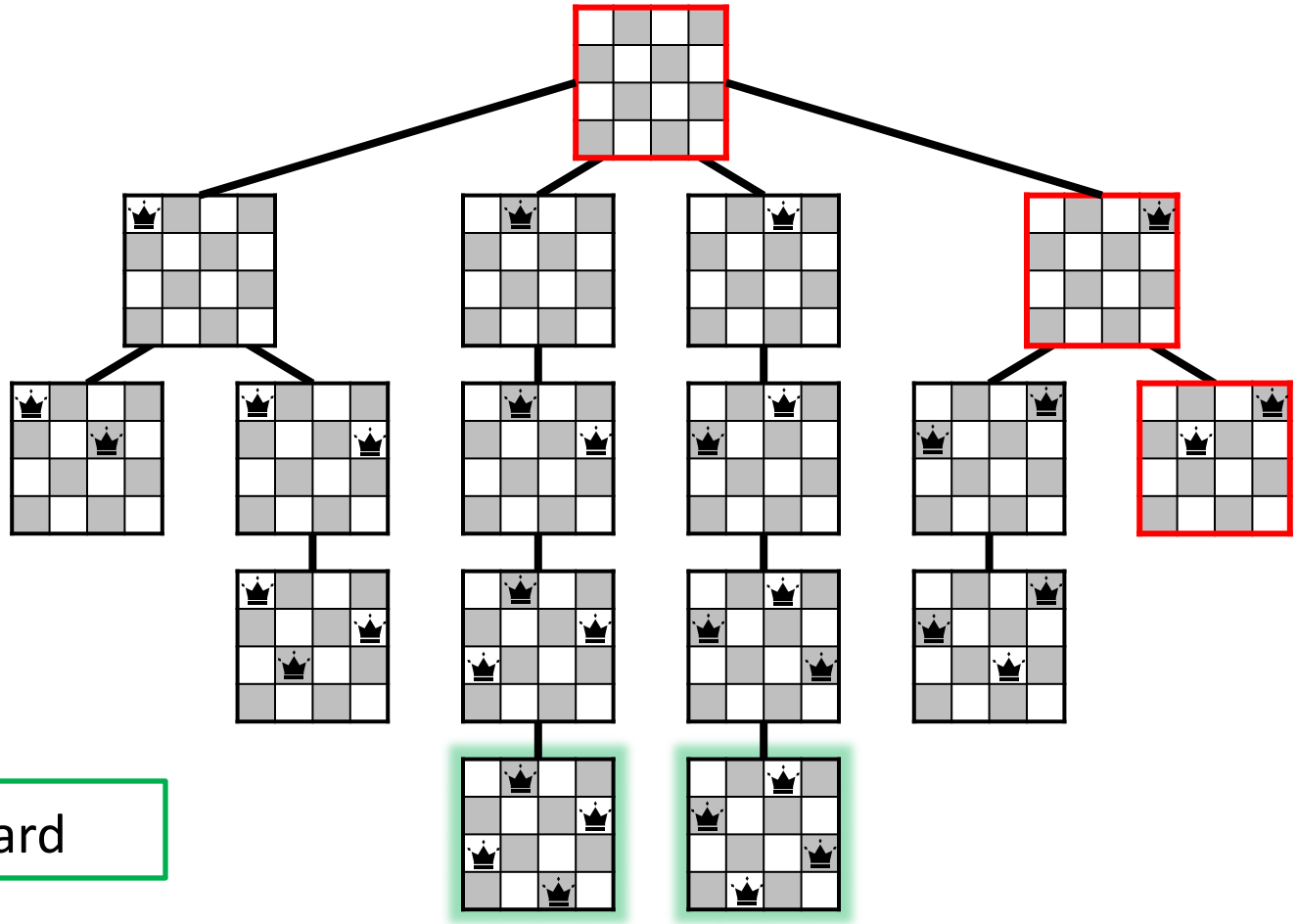
Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



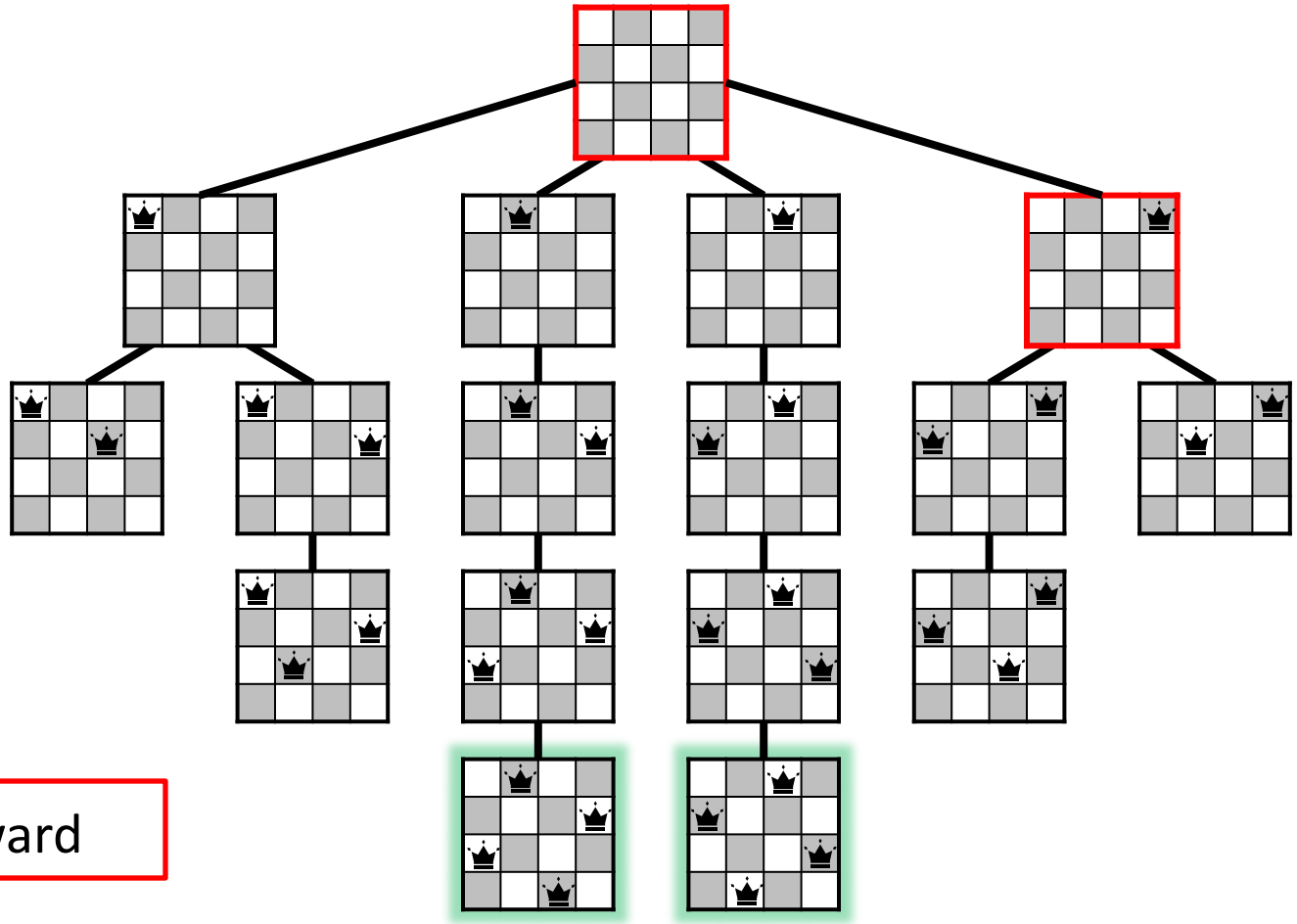
Forward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



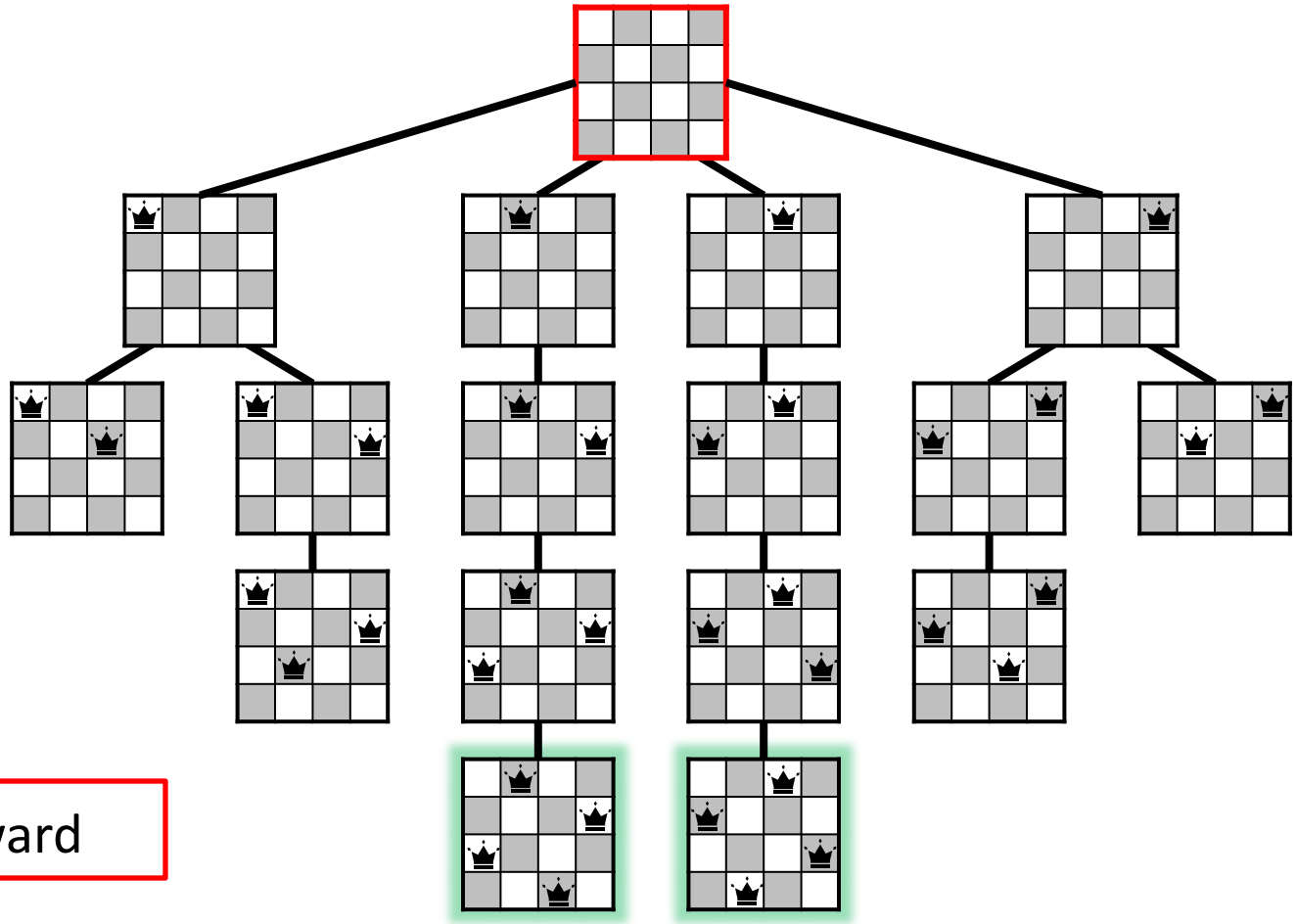
Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



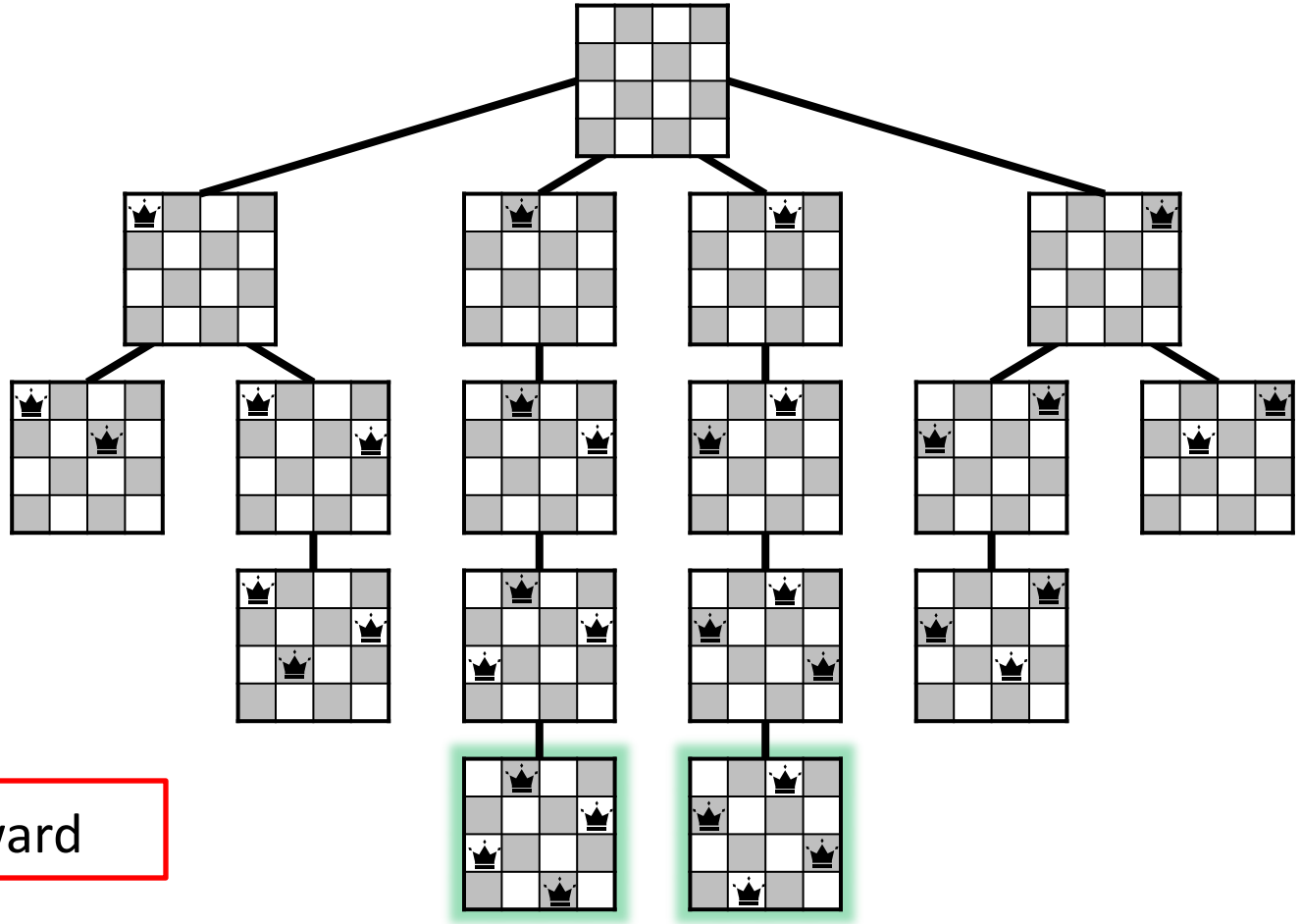
Backward

The n-Queens Problem

The following figure shows the full recursion tree for the 4-queens problem.

PLACE-QUEENS returns as soon as it finds a solution (the leftmost solution).

Modifying *PLACE-QUEENS* to return all solutions is easy.



Backward

The n-Queens Problem

Let $T(n, r)$ be the time needed to placing queens on rows r to n on a board of size $n \times n$.

Then $$T(n,r) \leq \begin{cases} \Theta(1), & if\ r > n, \\ n\big(T(n, r+1) + O(r)\big), & otherwise. \end{cases}$$

Solving, $T(n, r) = O(n^{n-r+1})$.

So, $T(n, 1) = O(n^n)$.

The Subset Sum Problem

Given an array $A[1..n]$ of n positive integers and a target integer T, determine if any subset of the numbers in A sum up to T.

The Subset Sum Problem

Given an array $A[1..n]$ of n positive integers and a target integer T, determine if any subset of the numbers in A sum up to T.

Let $S(i,t)$ be $True$ iff some subset of $A[i..n]$ adds up to t.

Then

$$S(i,t) = \begin{cases} True, & if\ t = 0, \\ False, & if\ t < 0\ or\ i > n, \\ S(i+1,t) \vee S(i+1,t-A[i]), & otherwise. \end{cases}$$

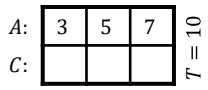
The Subset Sum Problem

Given an array $A[1..n]$ of n positive integers and a target integer T, determine if any subset of the numbers in A sum up to T.

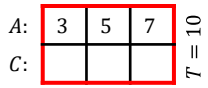
When called as SUBSET-SUM($A[1:n], C[1:n], n, T, 1$), the following algorithm returns a solution to the subset sum problem in $C[1:n]$, where for $1 \leq k \leq n$, $C[k] =$ TRUE if $A[k]$ is included in the solution, and FALSE otherwise.

```
SUBSET-SUM ( A, C, n, T, k )
1.      if T = 0 then
2.          return TRUE
3.      else if T < 0 or k > n then
4.          return FALSE
5.      else
6.          C[k] ← TRUE
7.          if SUBSET-SUM ( A, C, n, T − A[k], k + 1 ) = TRUE then
8.              return TRUE
9.          C[k] ← FALSE
10.         if SUBSET-SUM ( A, C, n, T, k + 1 ) = TRUE then
11.             return TRUE
12.         return FALSE
```

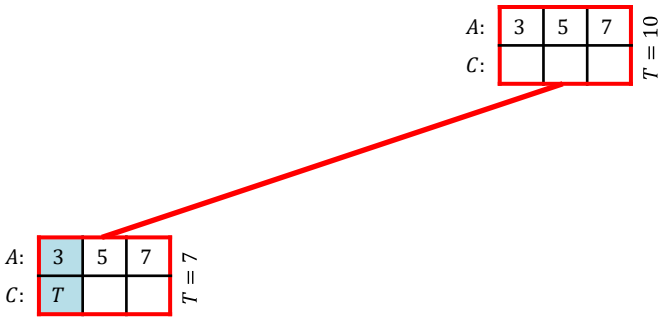
The Subset Sum Problem



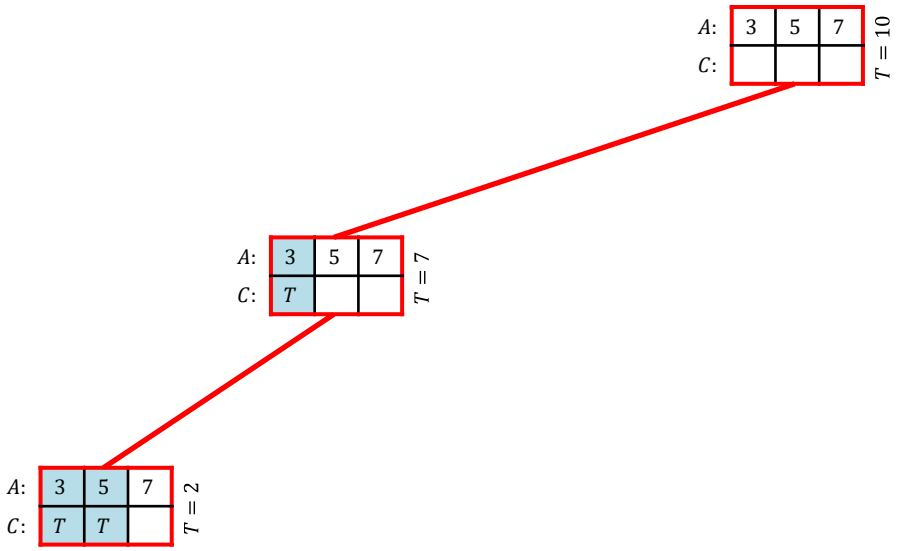
The Subset Sum Problem



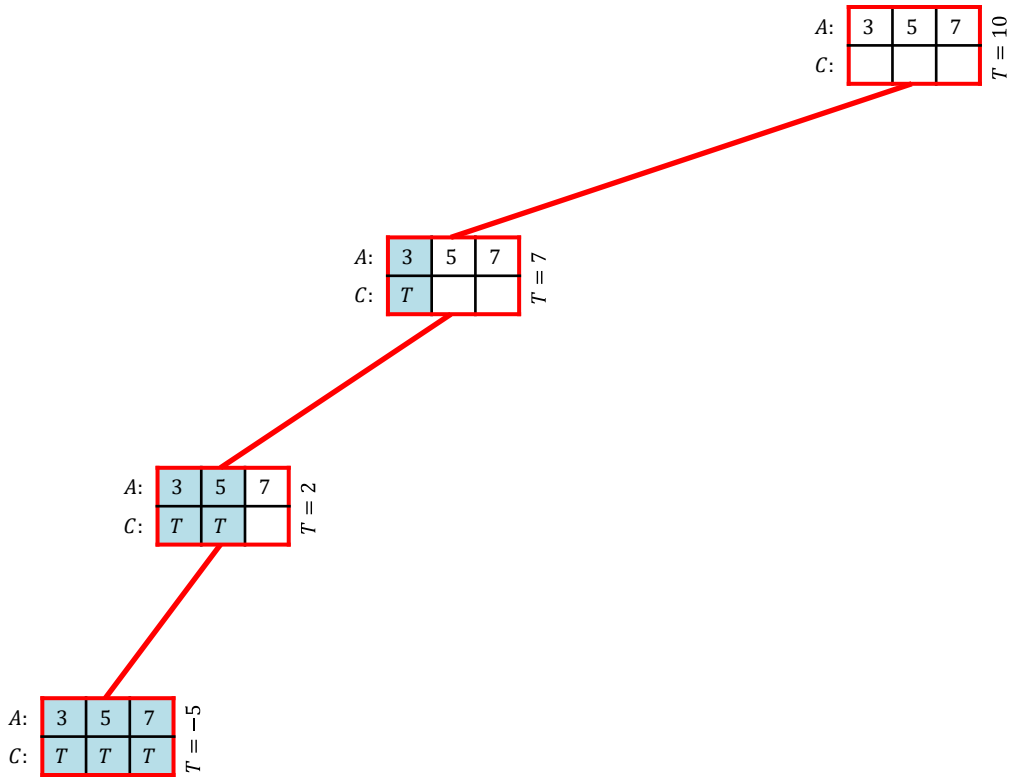
The Subset Sum Problem



The Subset Sum Problem

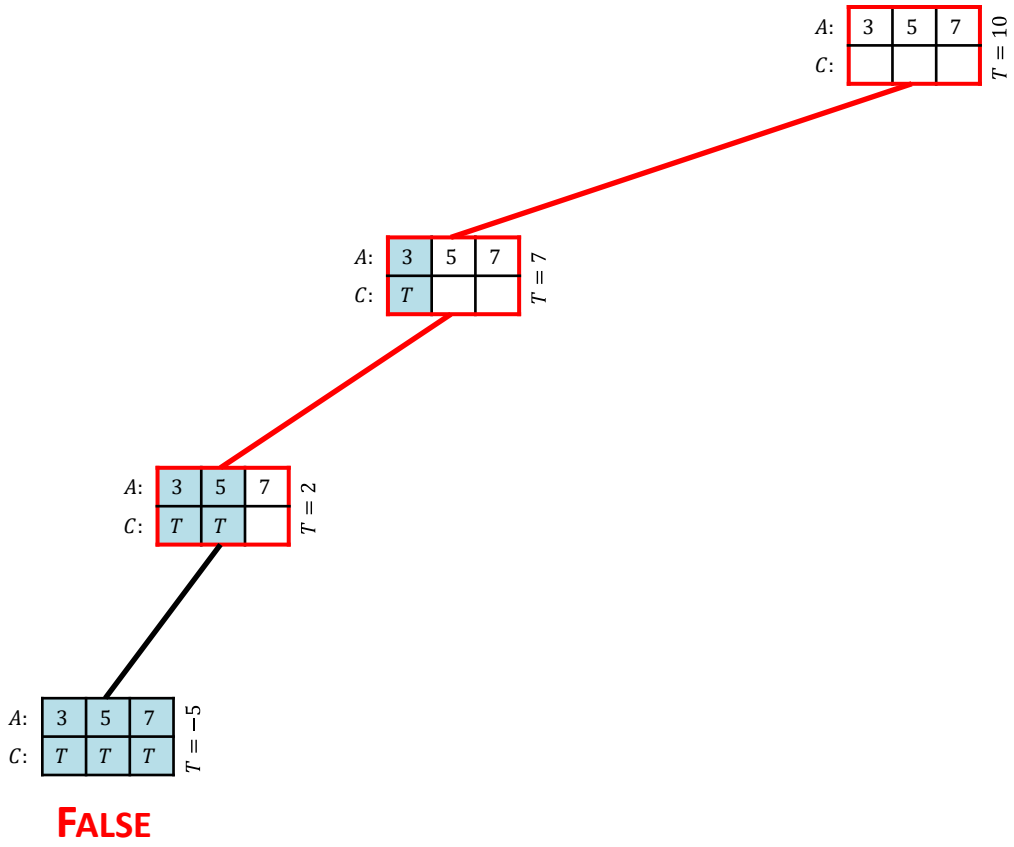


The Subset Sum Problem



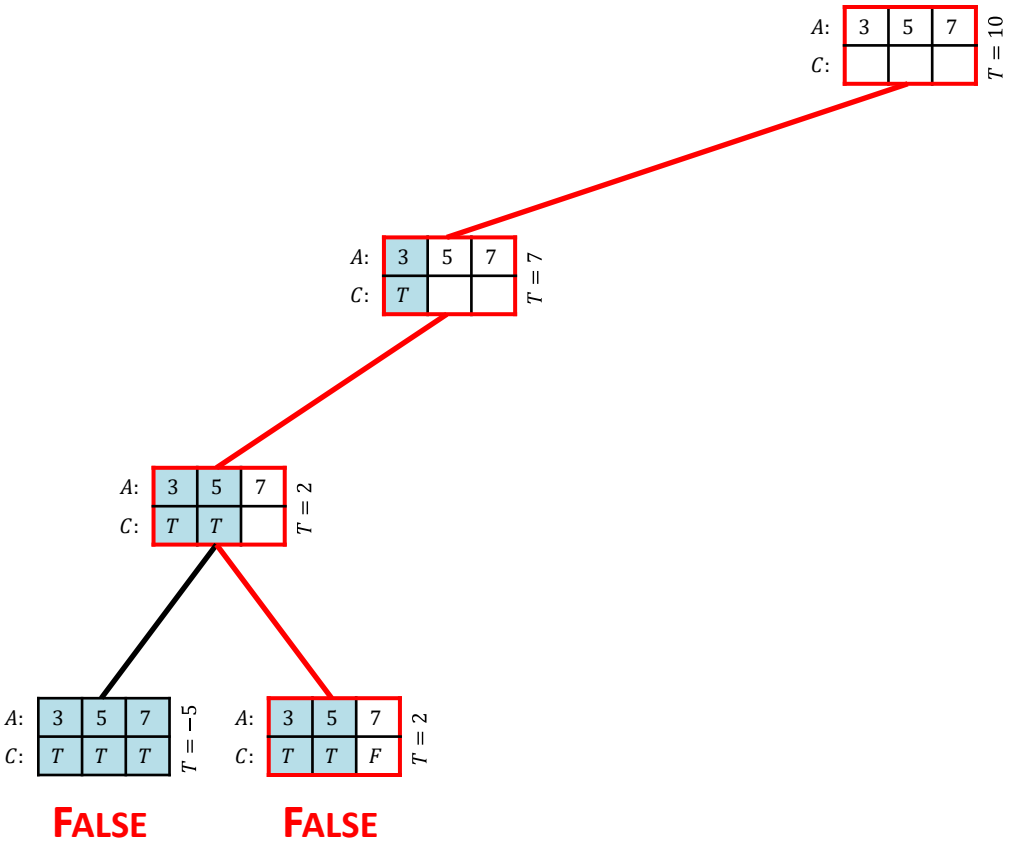
FALSE

The Subset Sum Problem

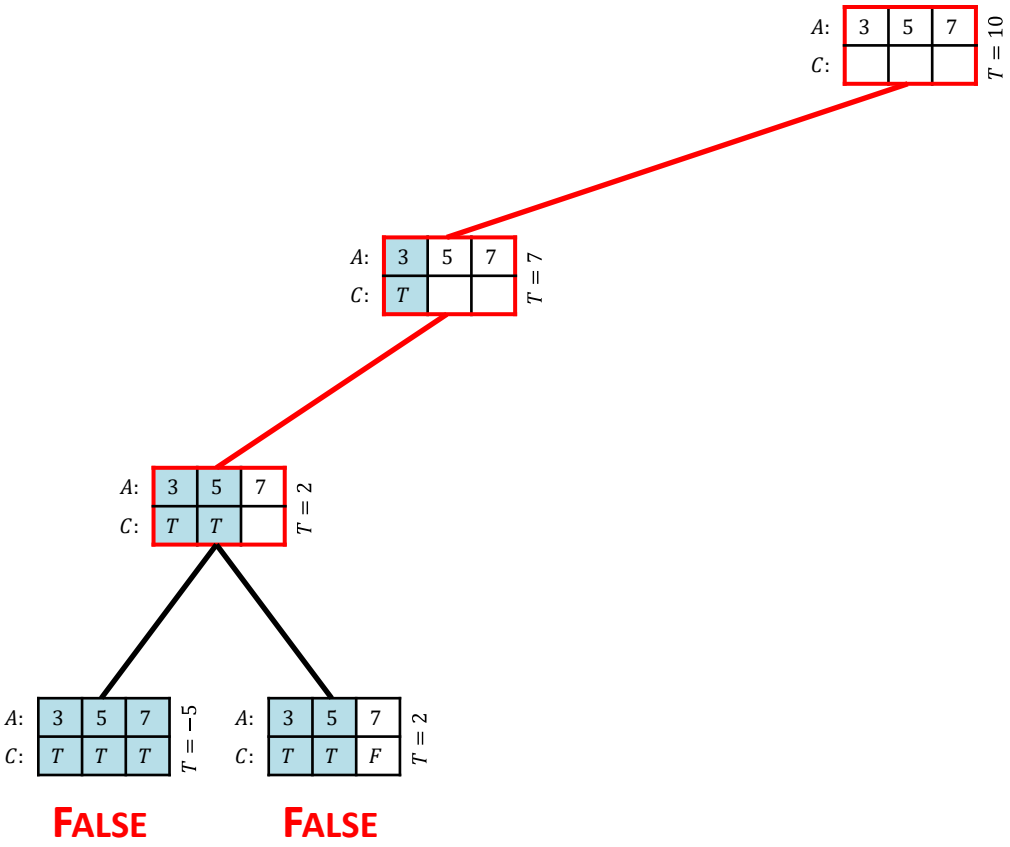


FALSE

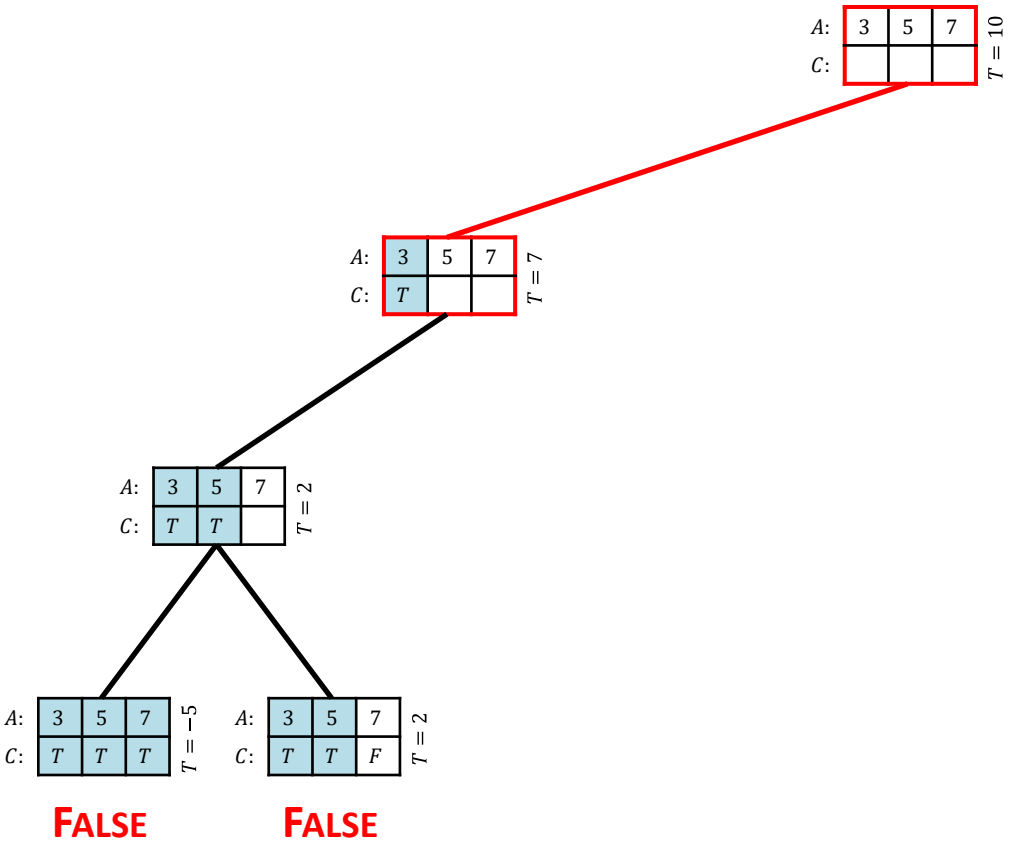
The Subset Sum Problem



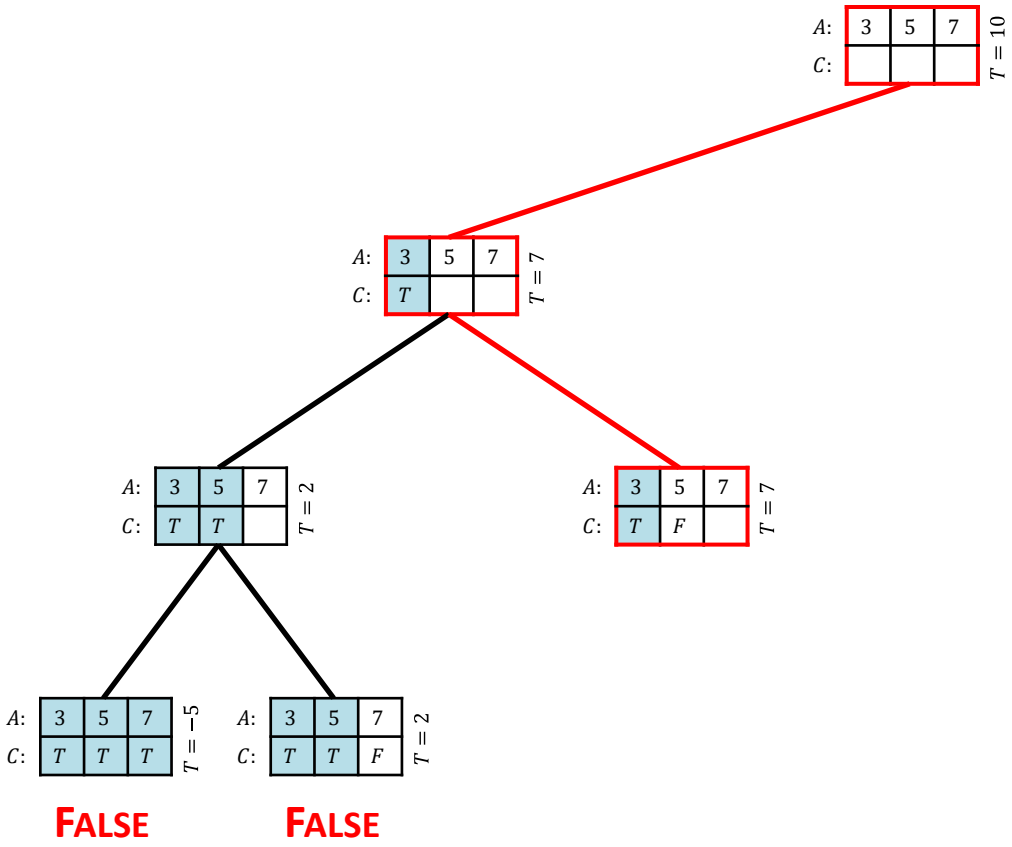
The Subset Sum Problem



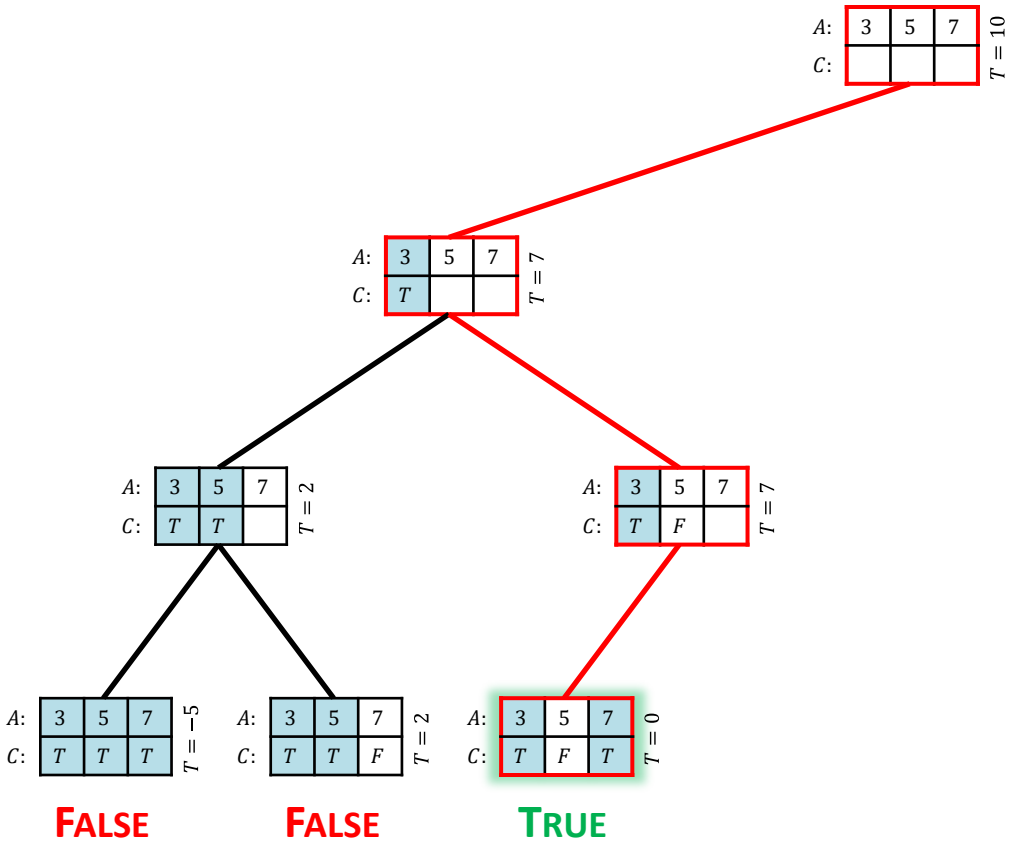
The Subset Sum Problem



The Subset Sum Problem

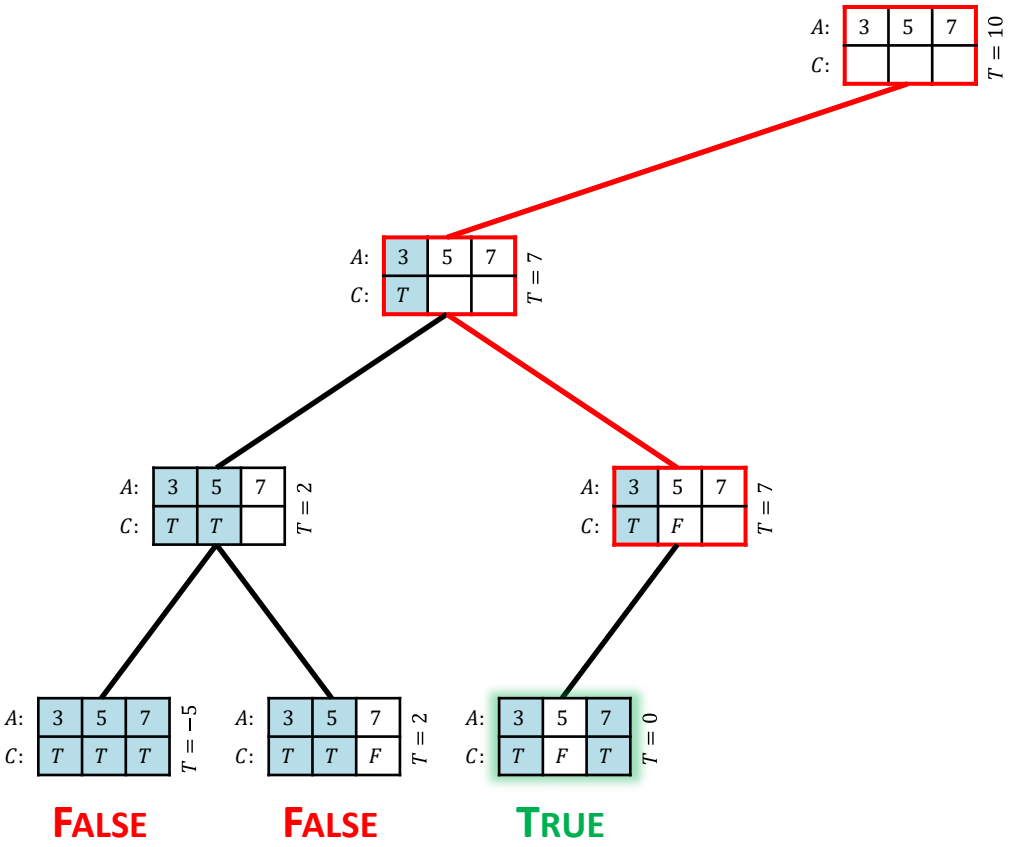


The Subset Sum Problem

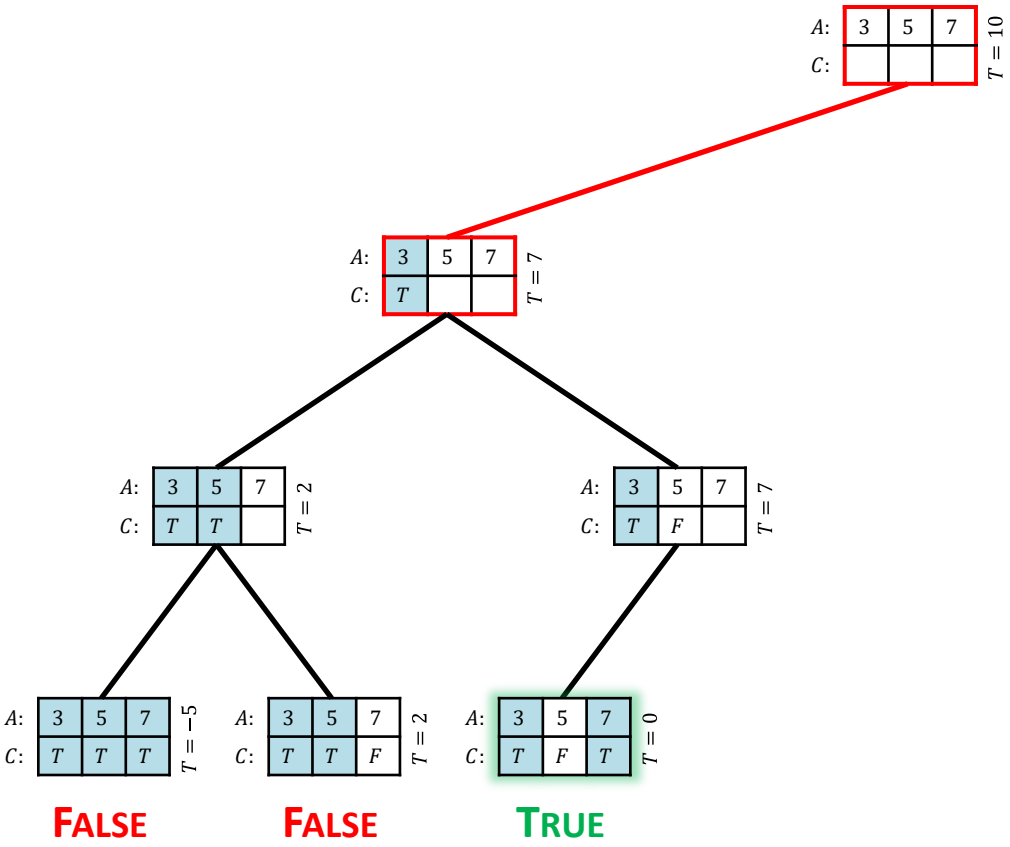


Solution Found!

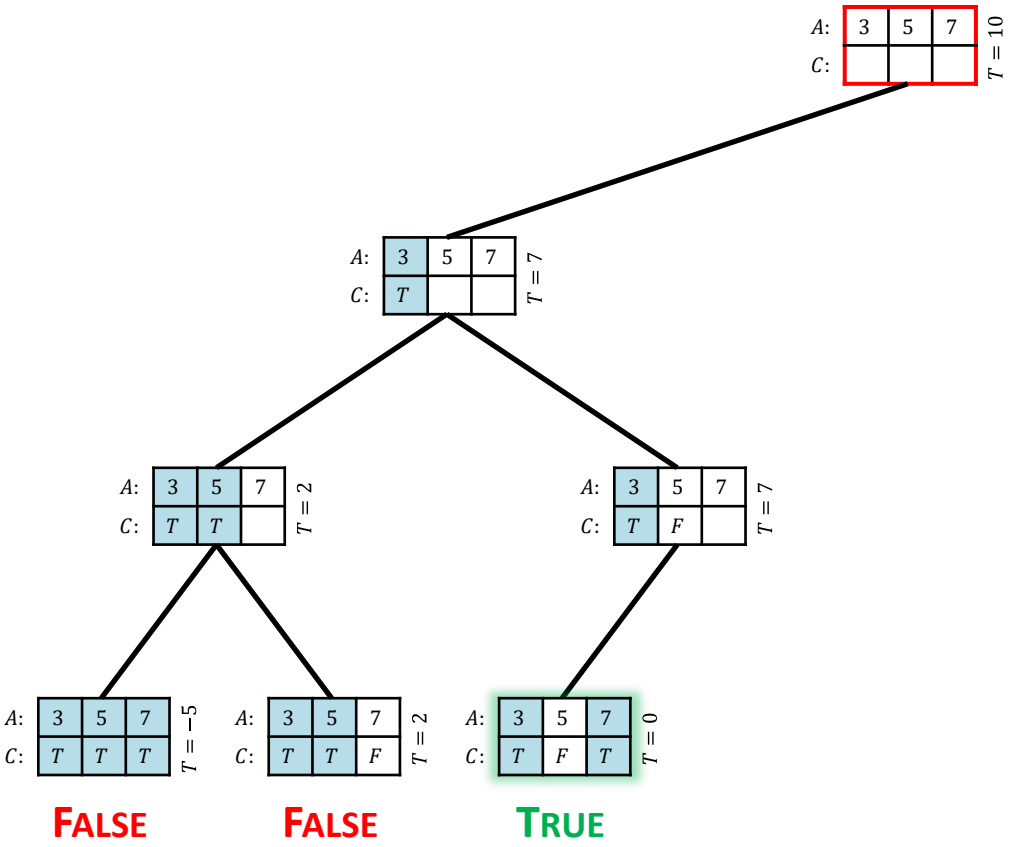
The Subset Sum Problem



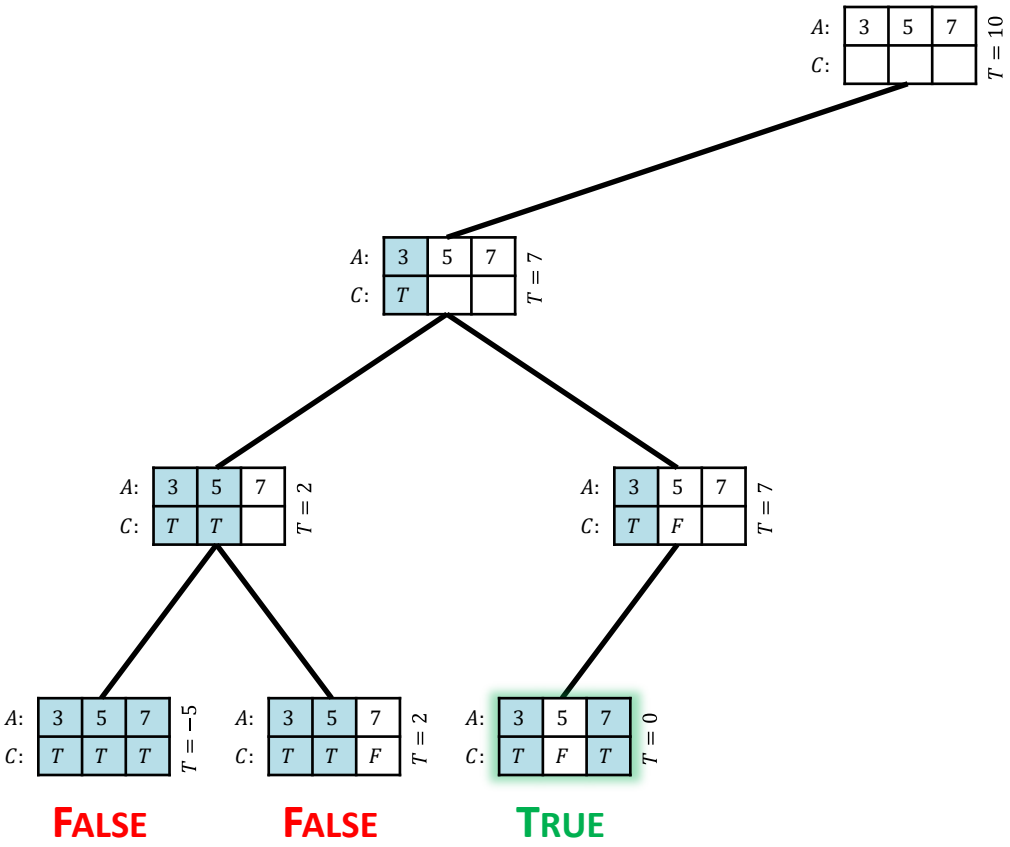
The Subset Sum Problem



The Subset Sum Problem

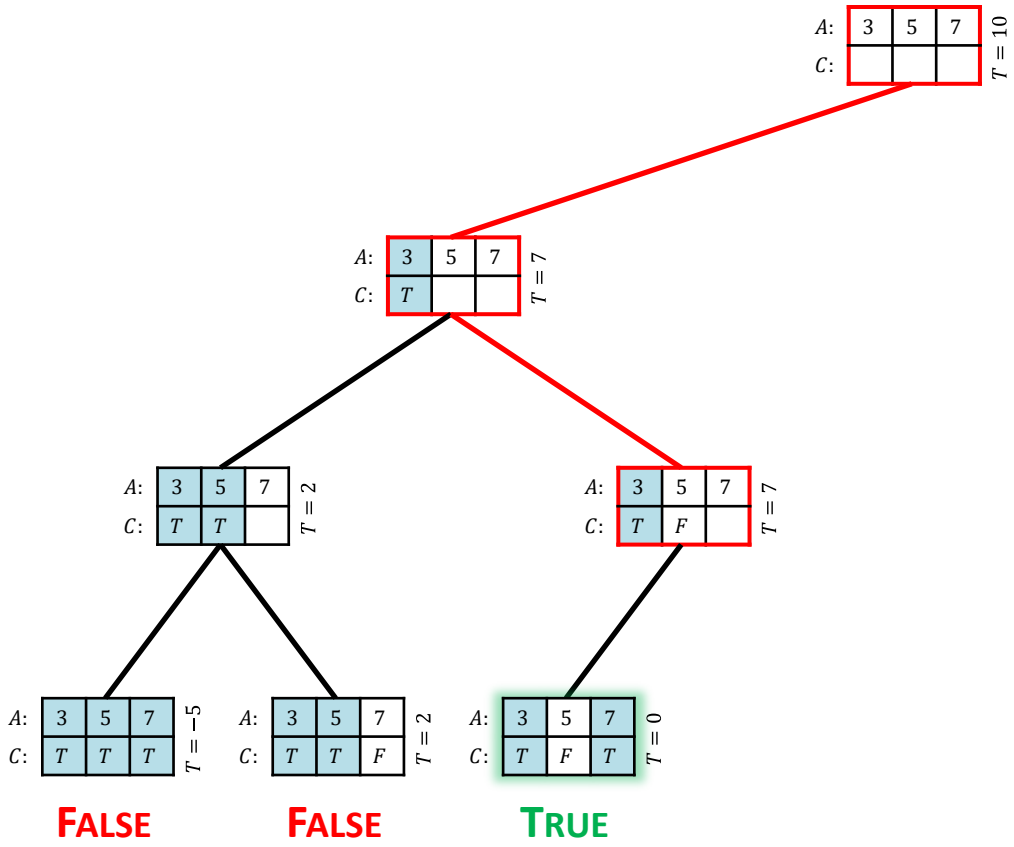


The Subset Sum Problem



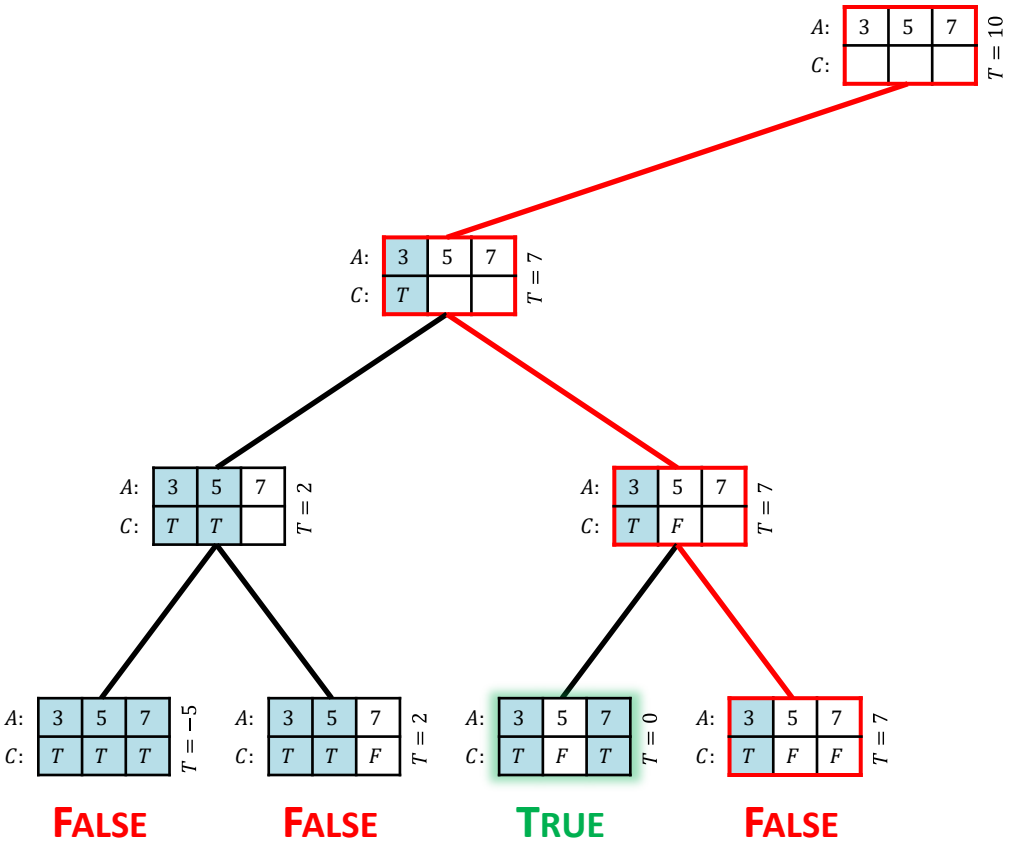
Done!

The Subset Sum Problem

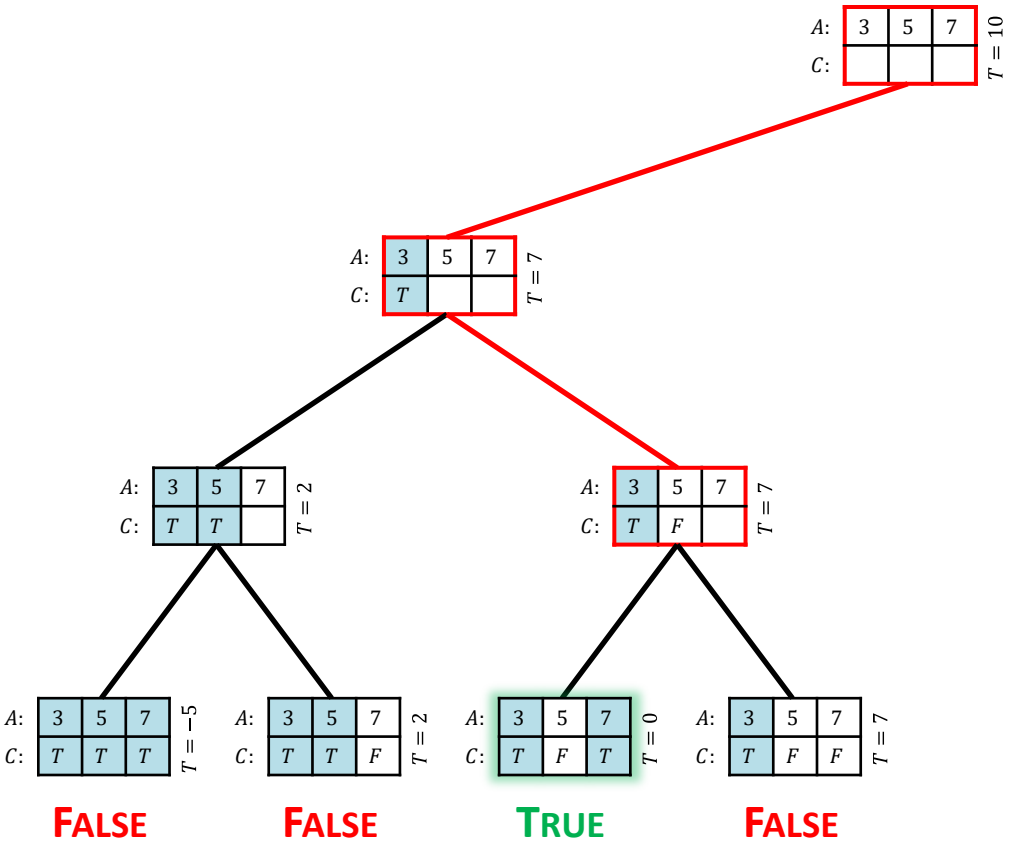


But Could Keep Going for all Solutions!

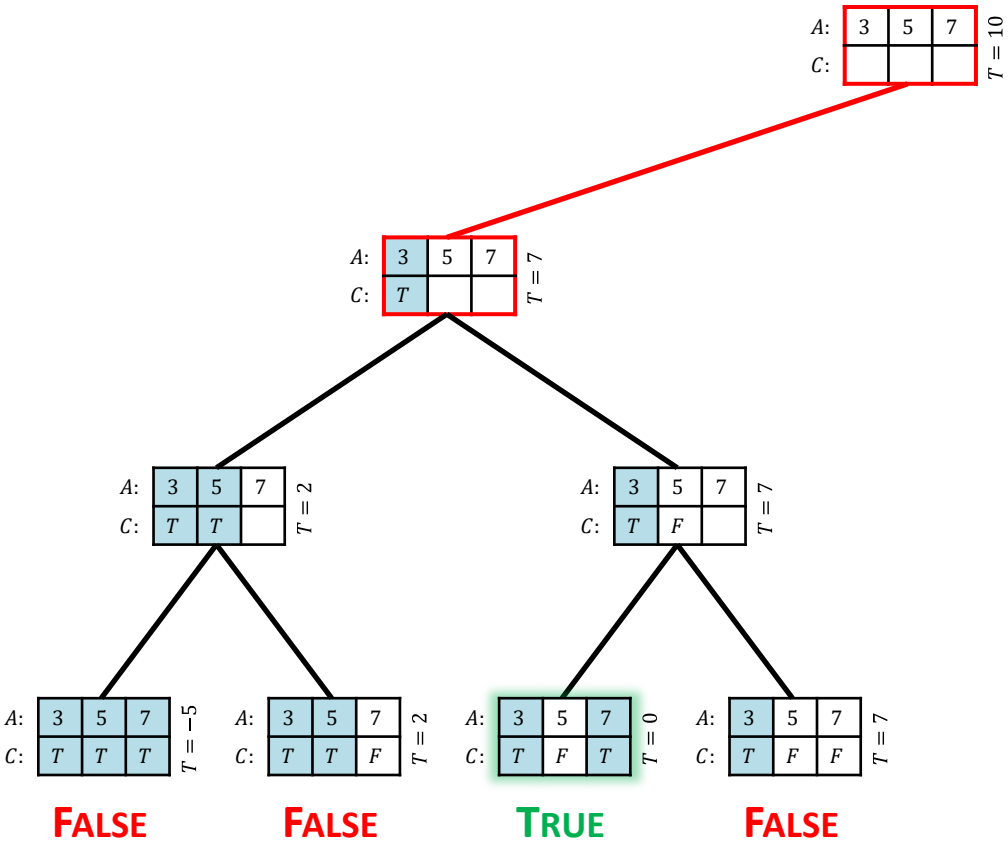
The Subset Sum Problem



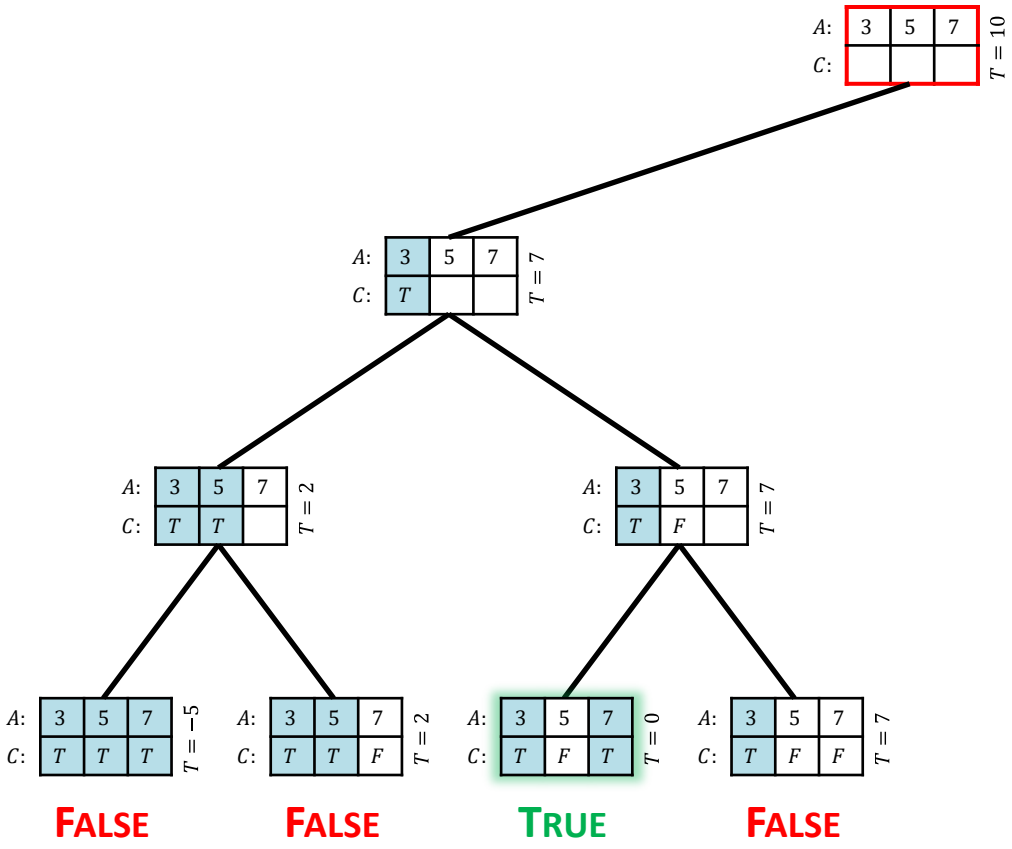
The Subset Sum Problem



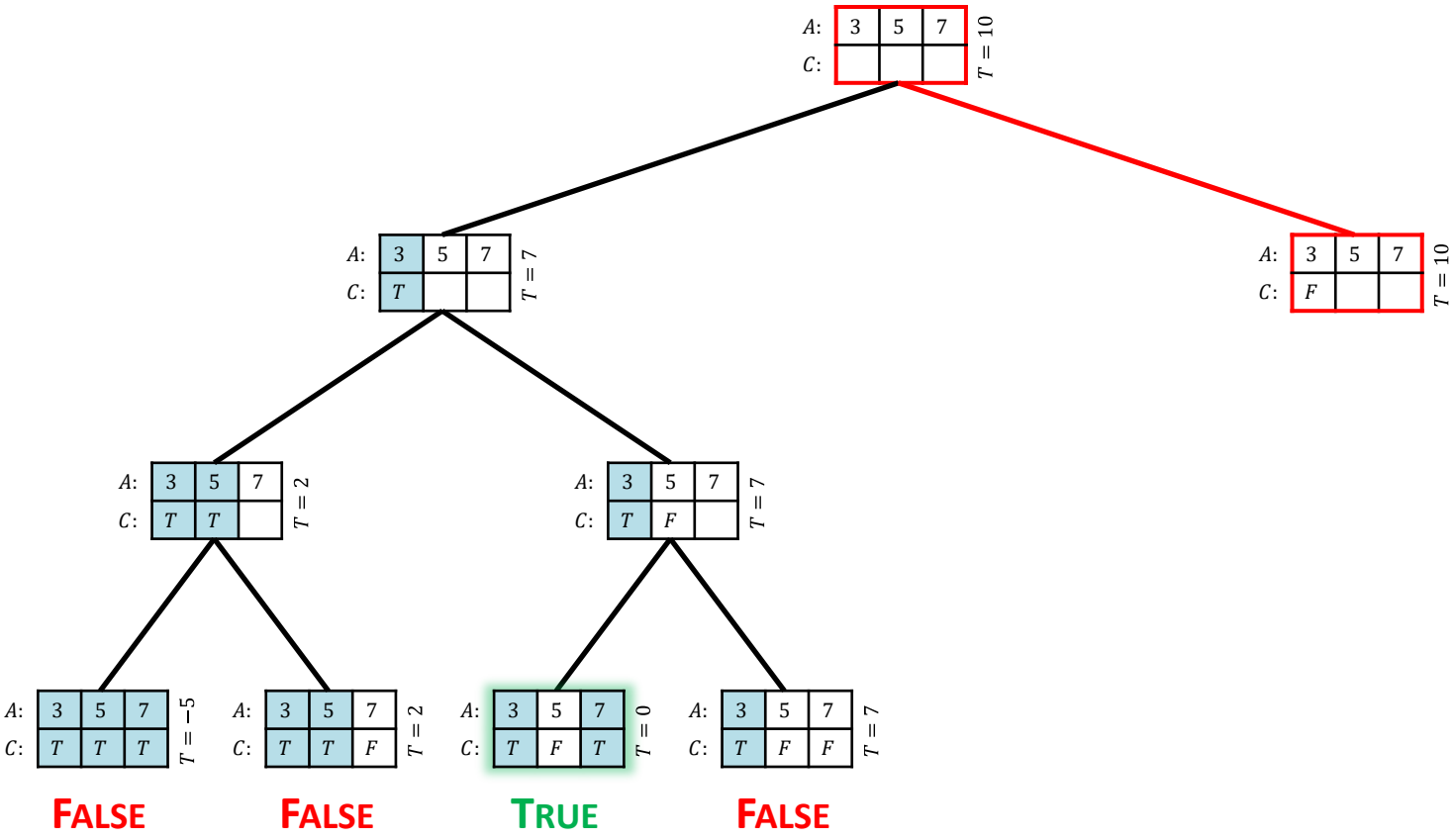
The Subset Sum Problem



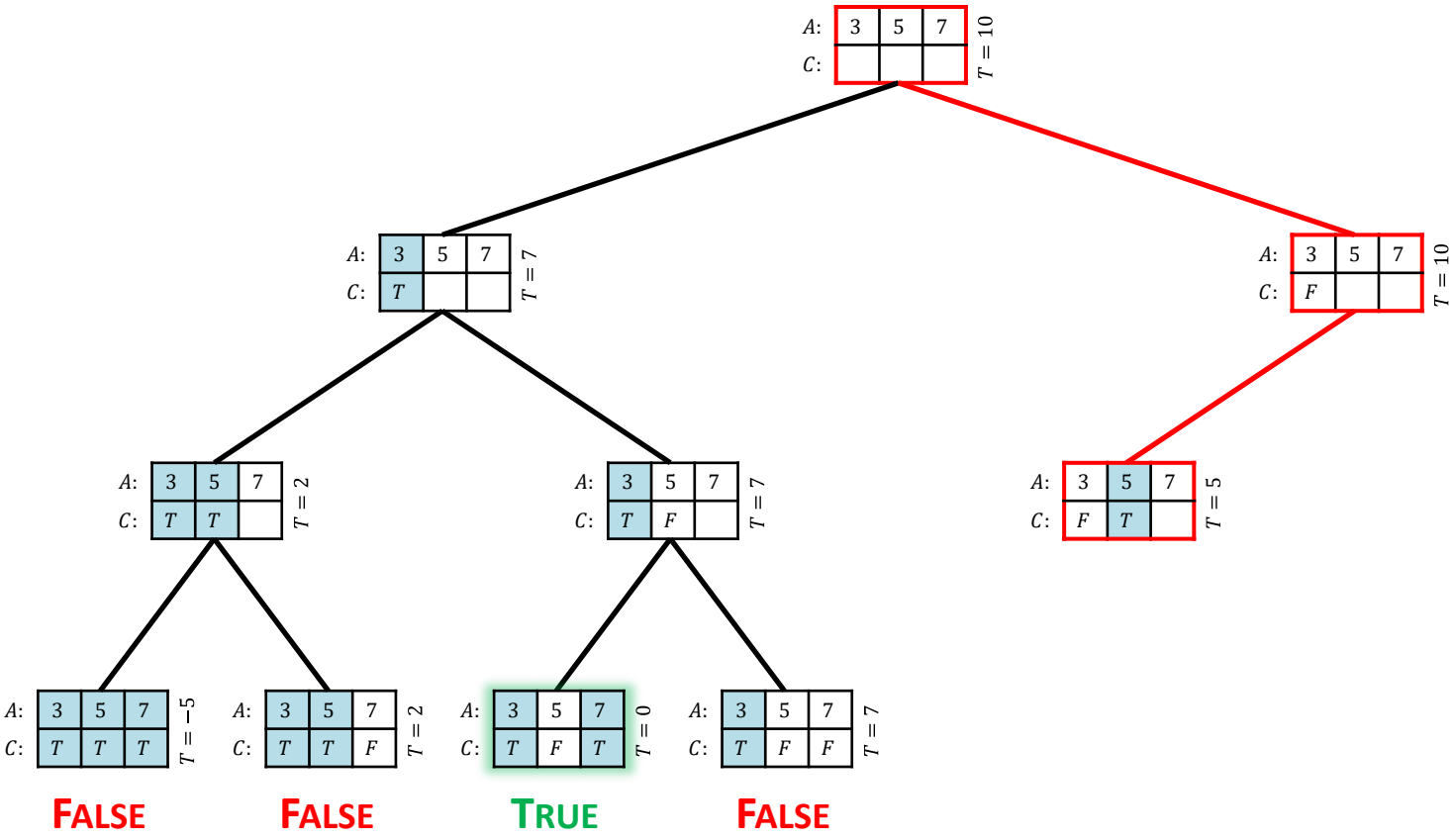
The Subset Sum Problem



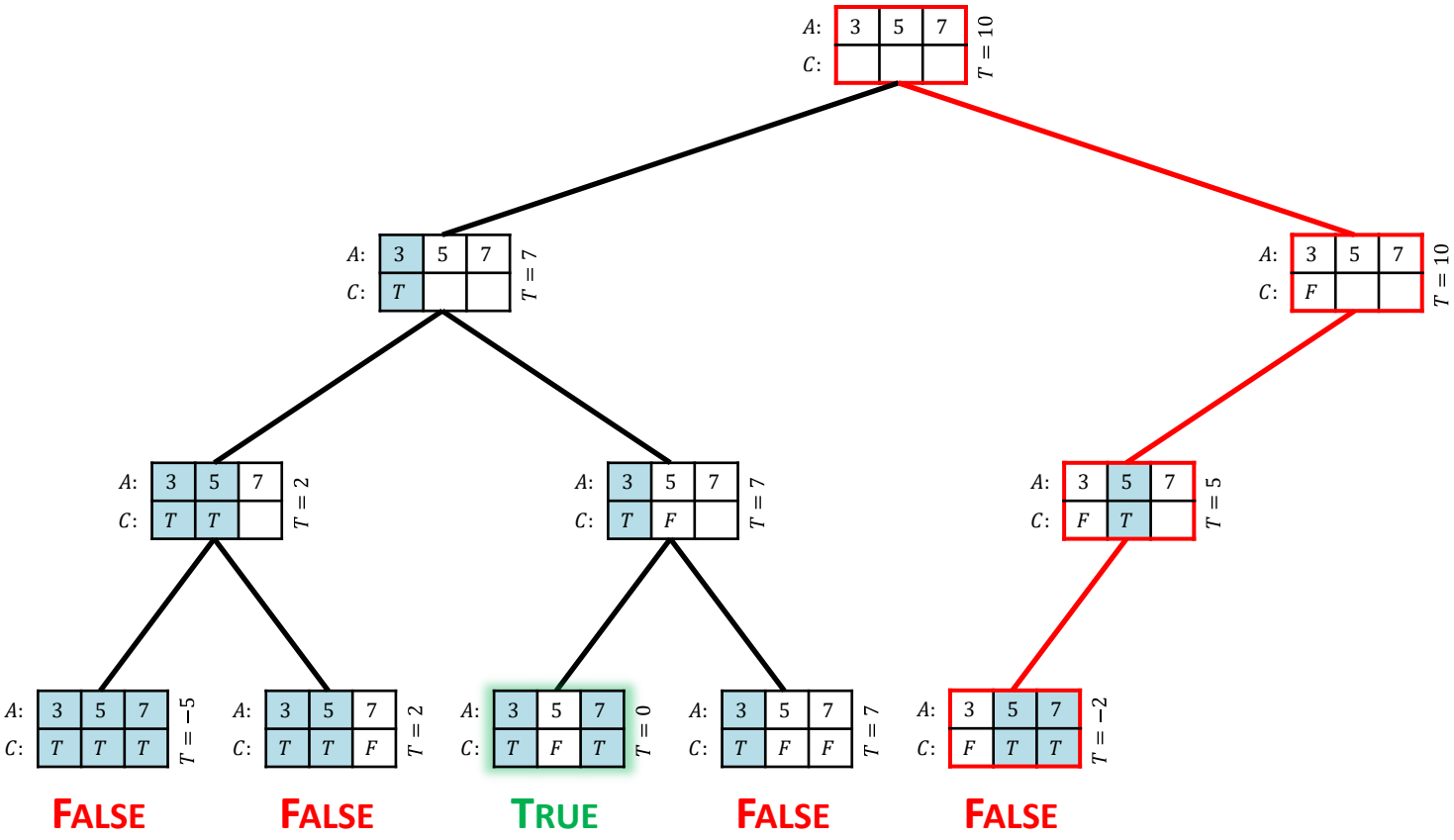
The Subset Sum Problem



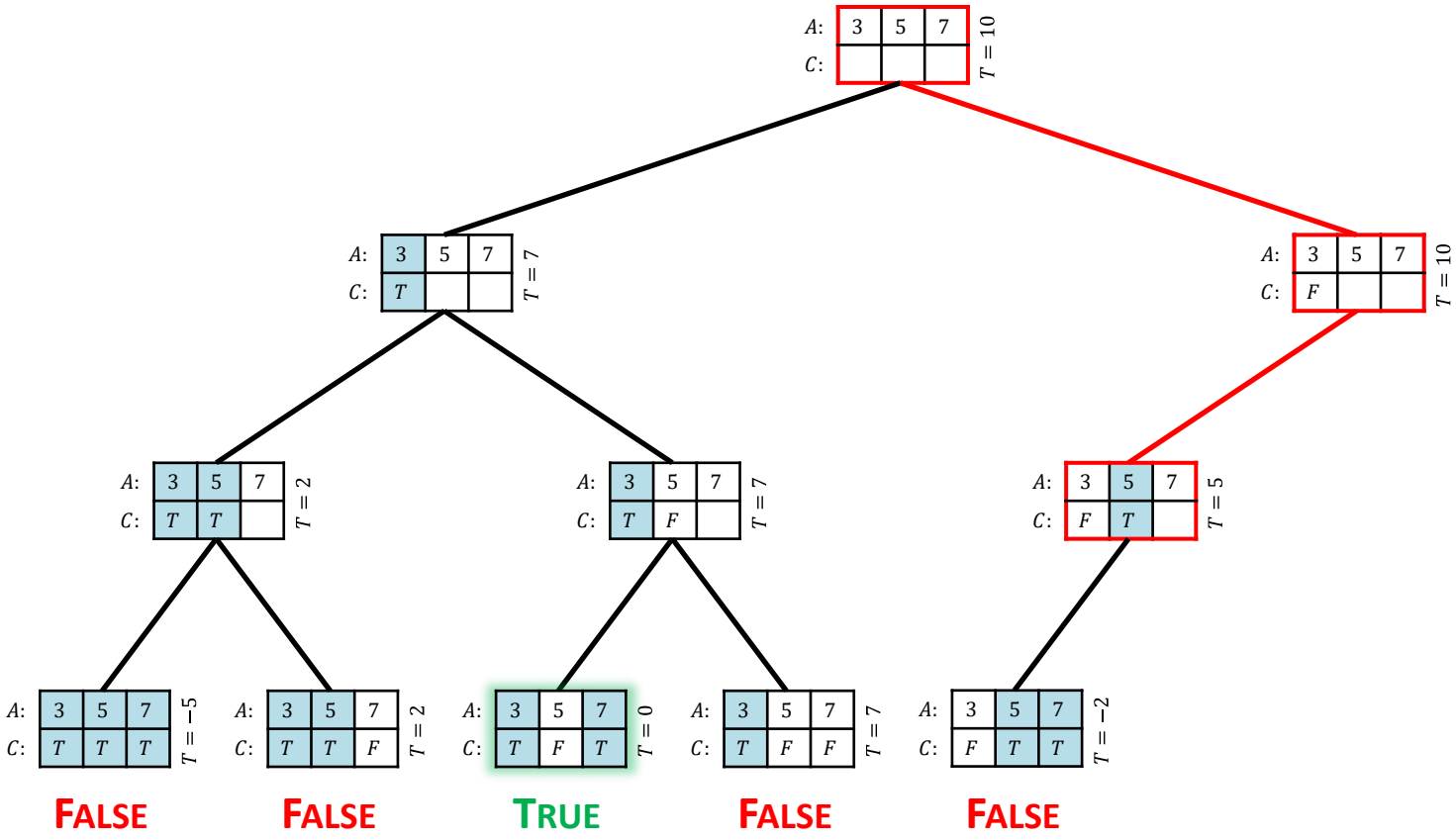
The Subset Sum Problem



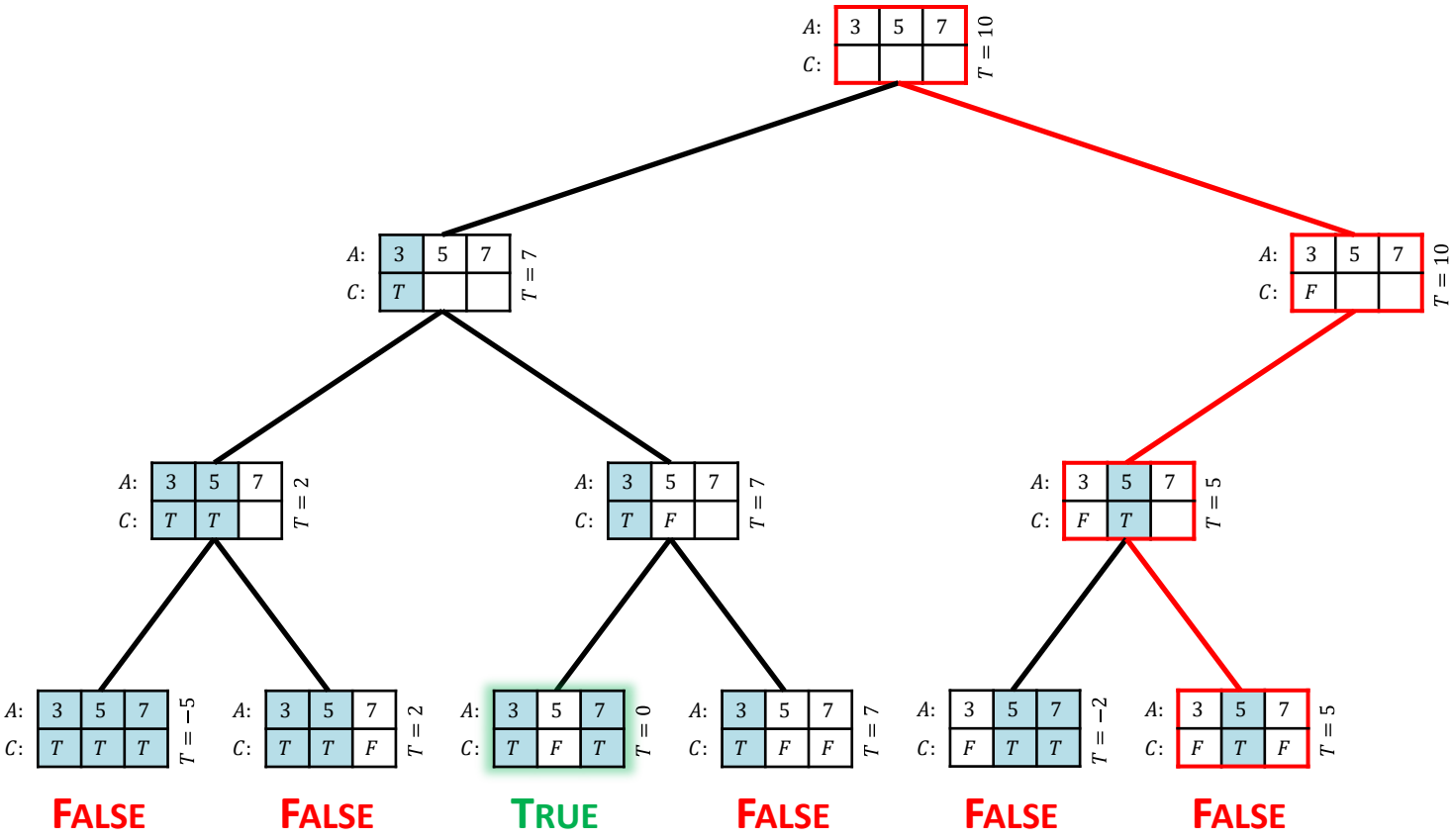
The Subset Sum Problem



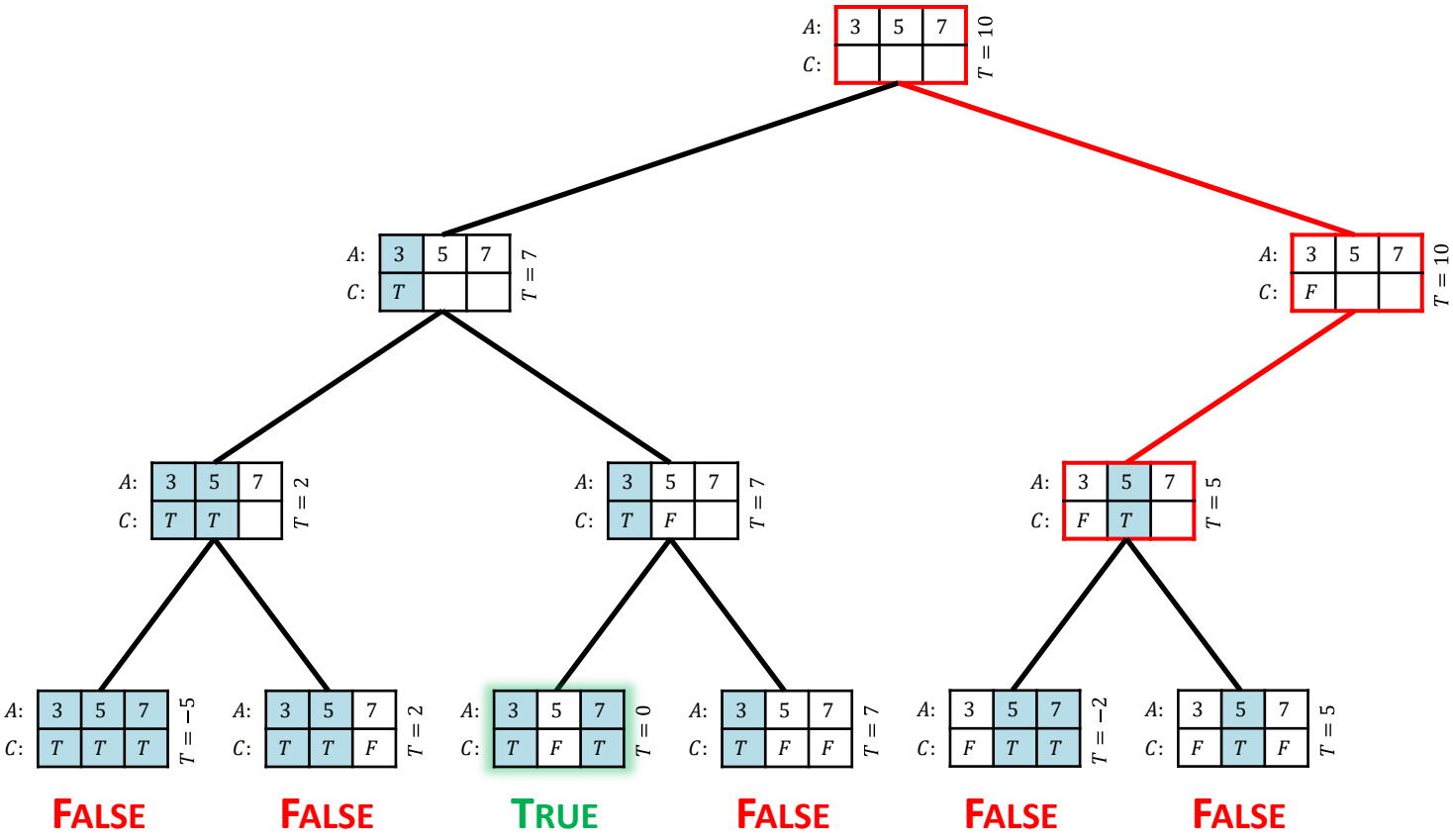
The Subset Sum Problem



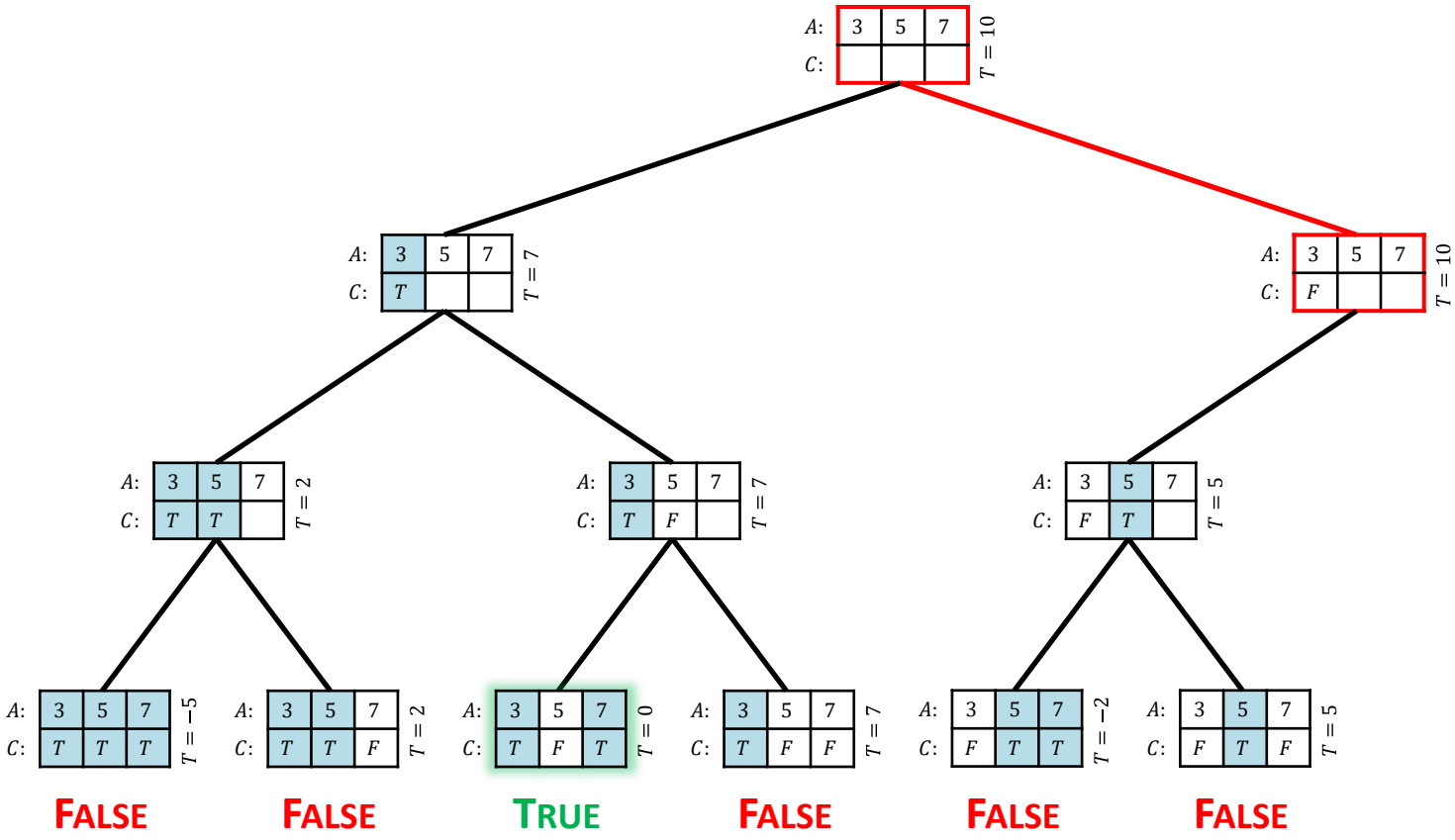
The Subset Sum Problem



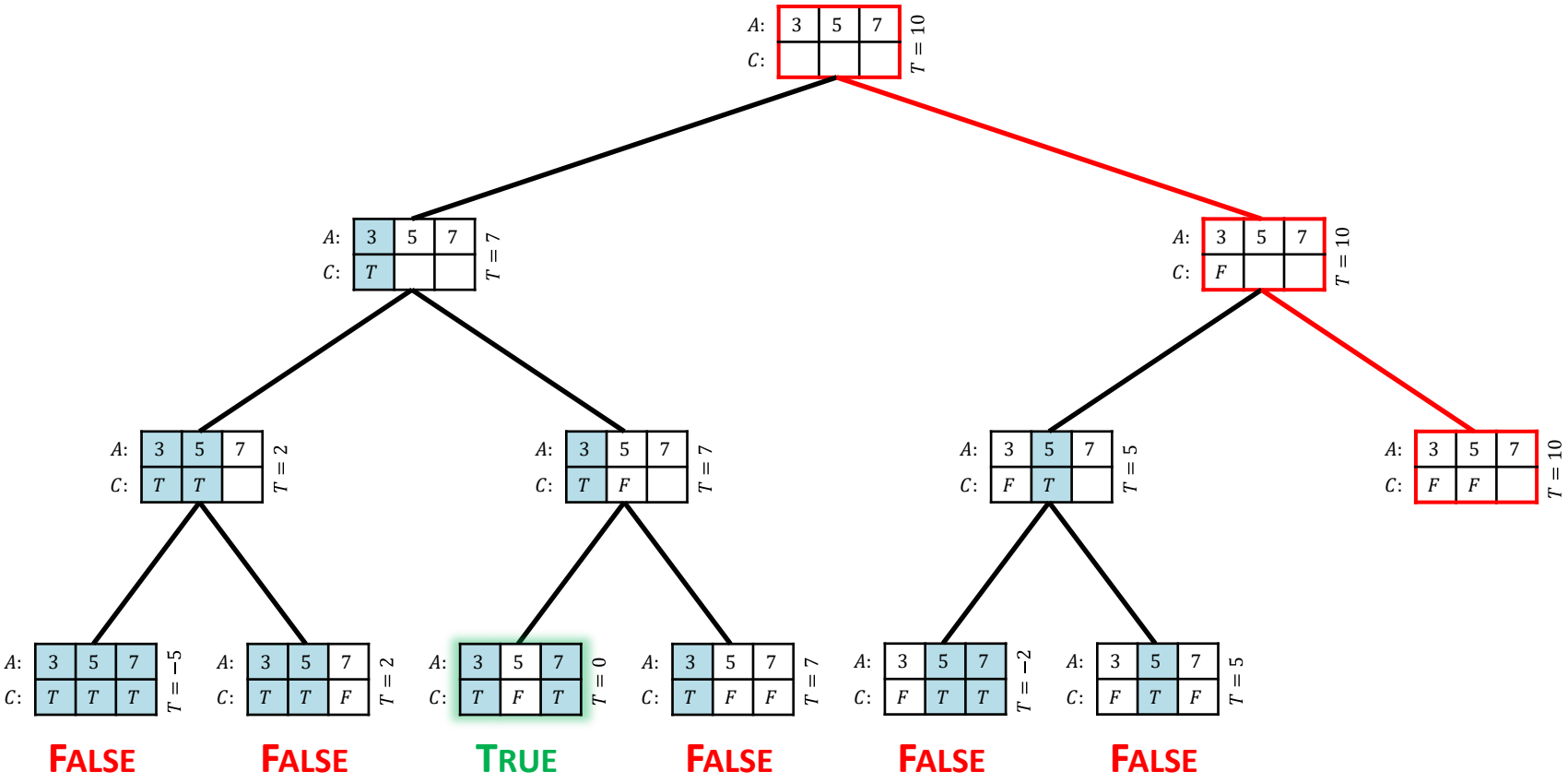
The Subset Sum Problem



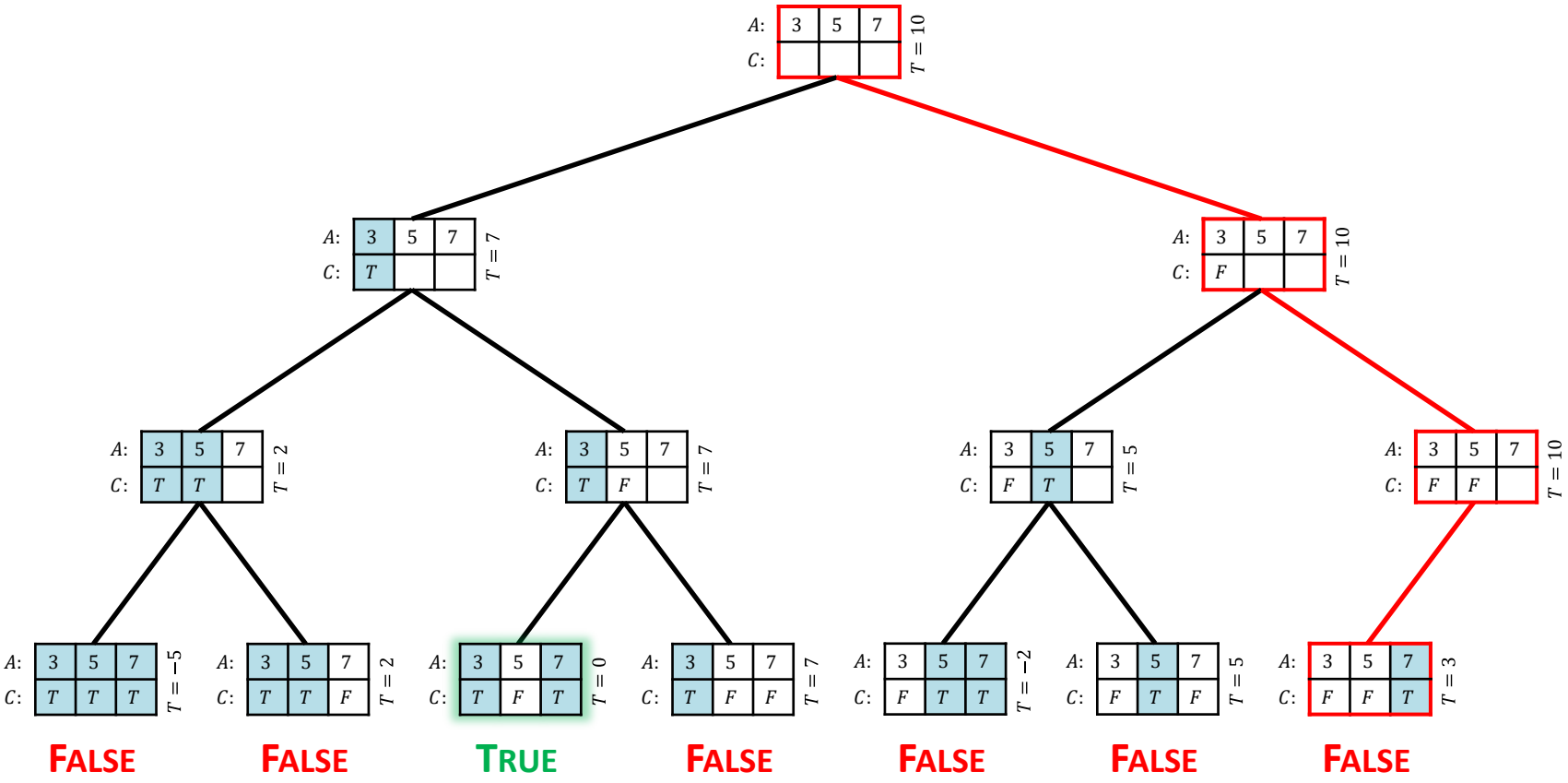
The Subset Sum Problem



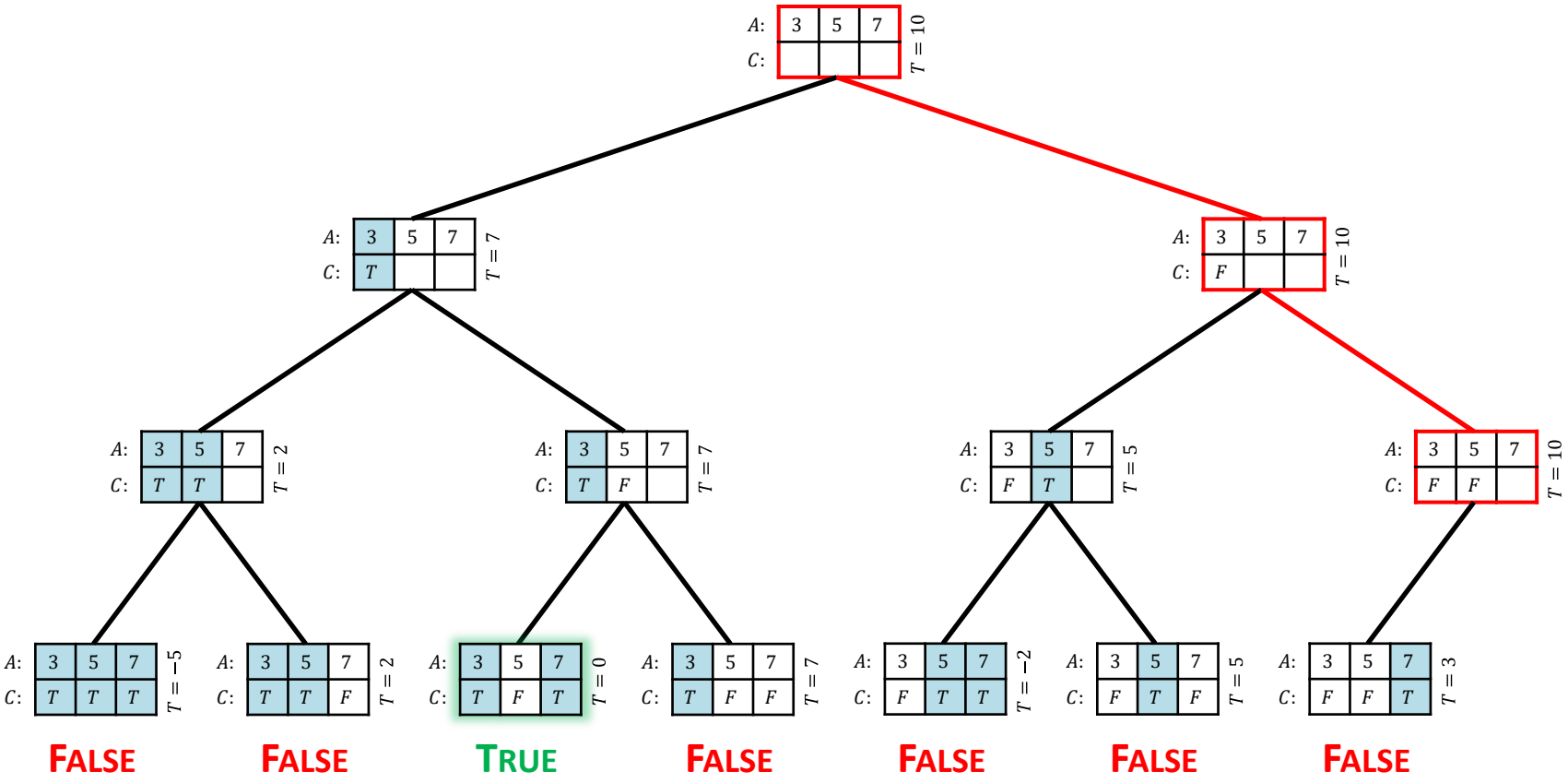
The Subset Sum Problem



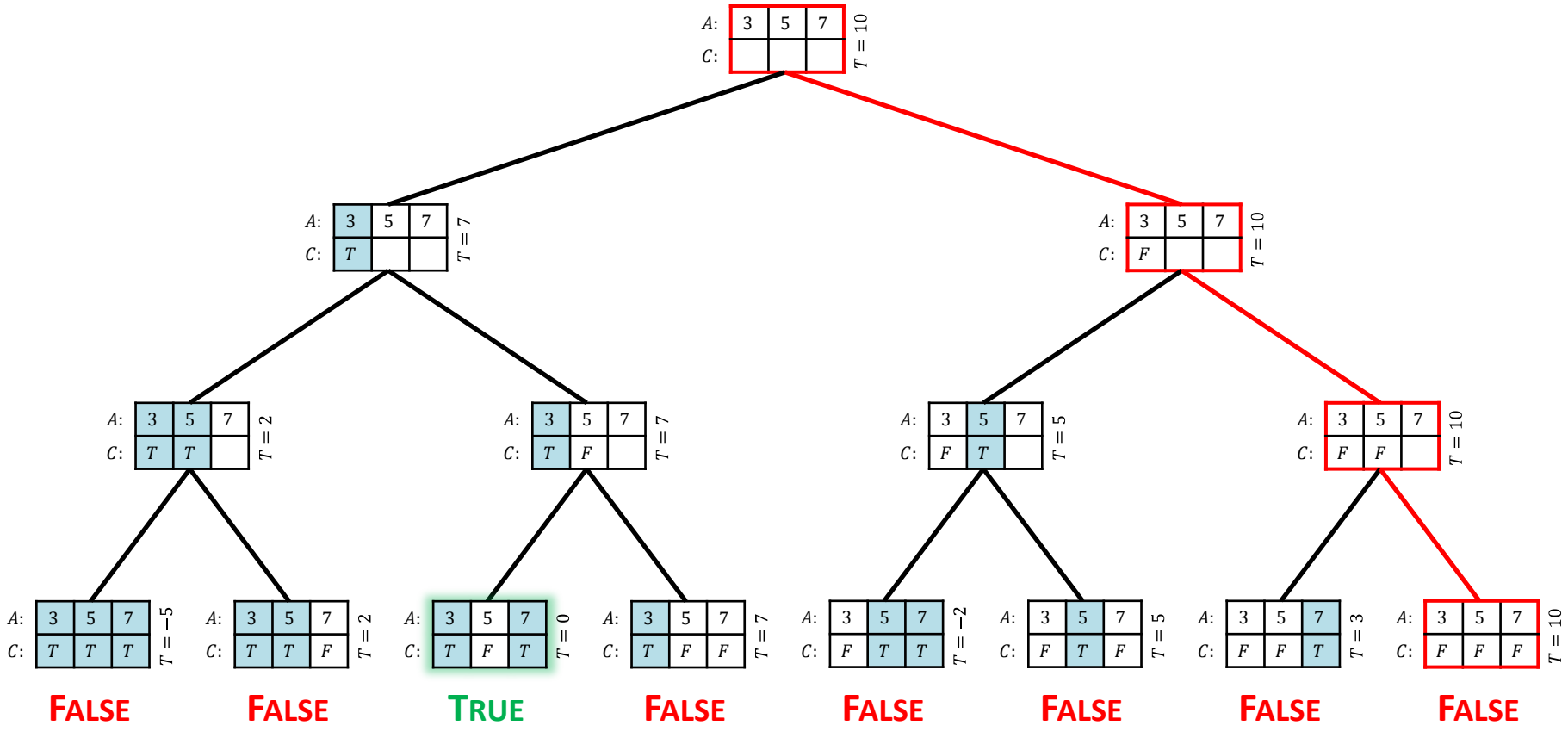
The Subset Sum Problem



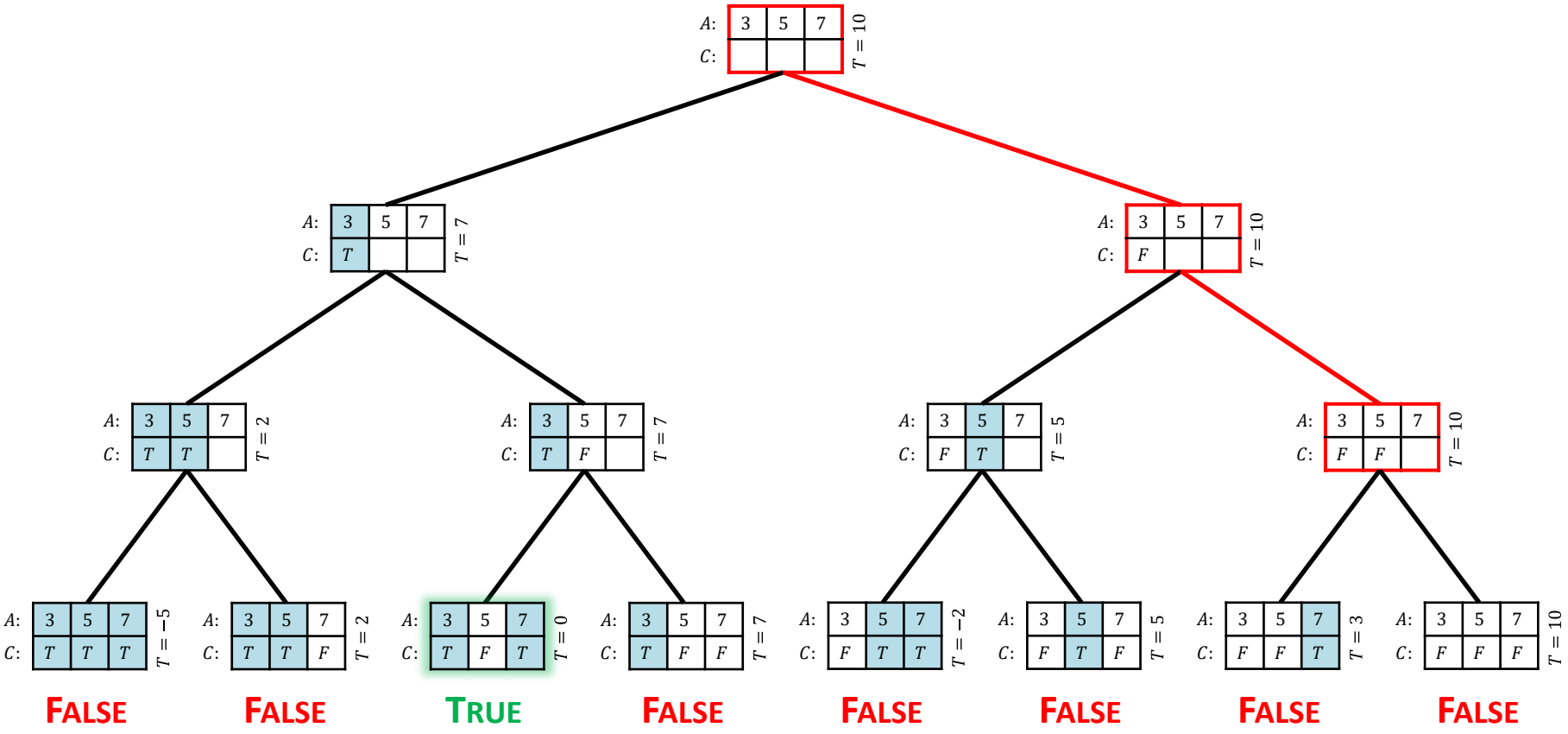
The Subset Sum Problem



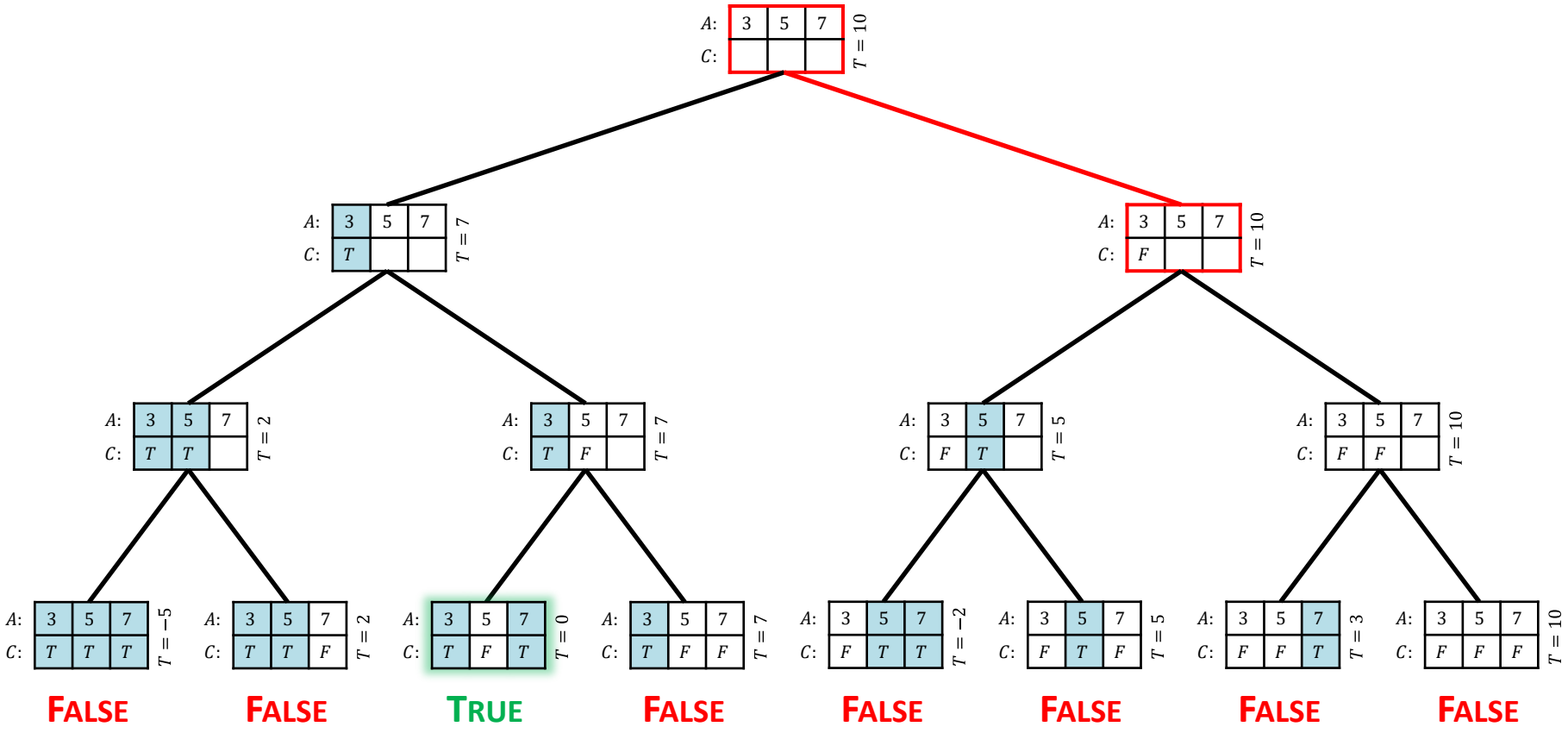
The Subset Sum Problem



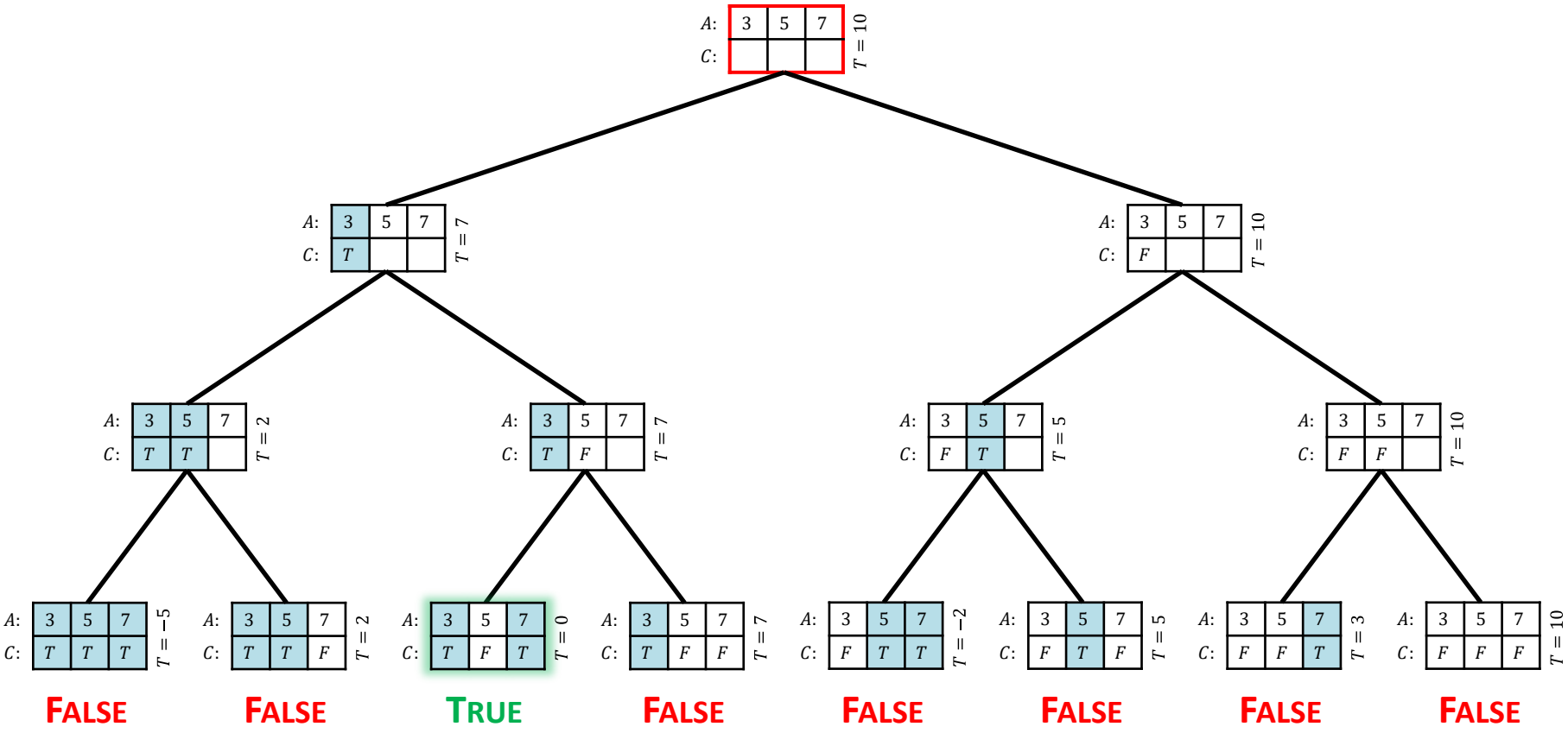
The Subset Sum Problem



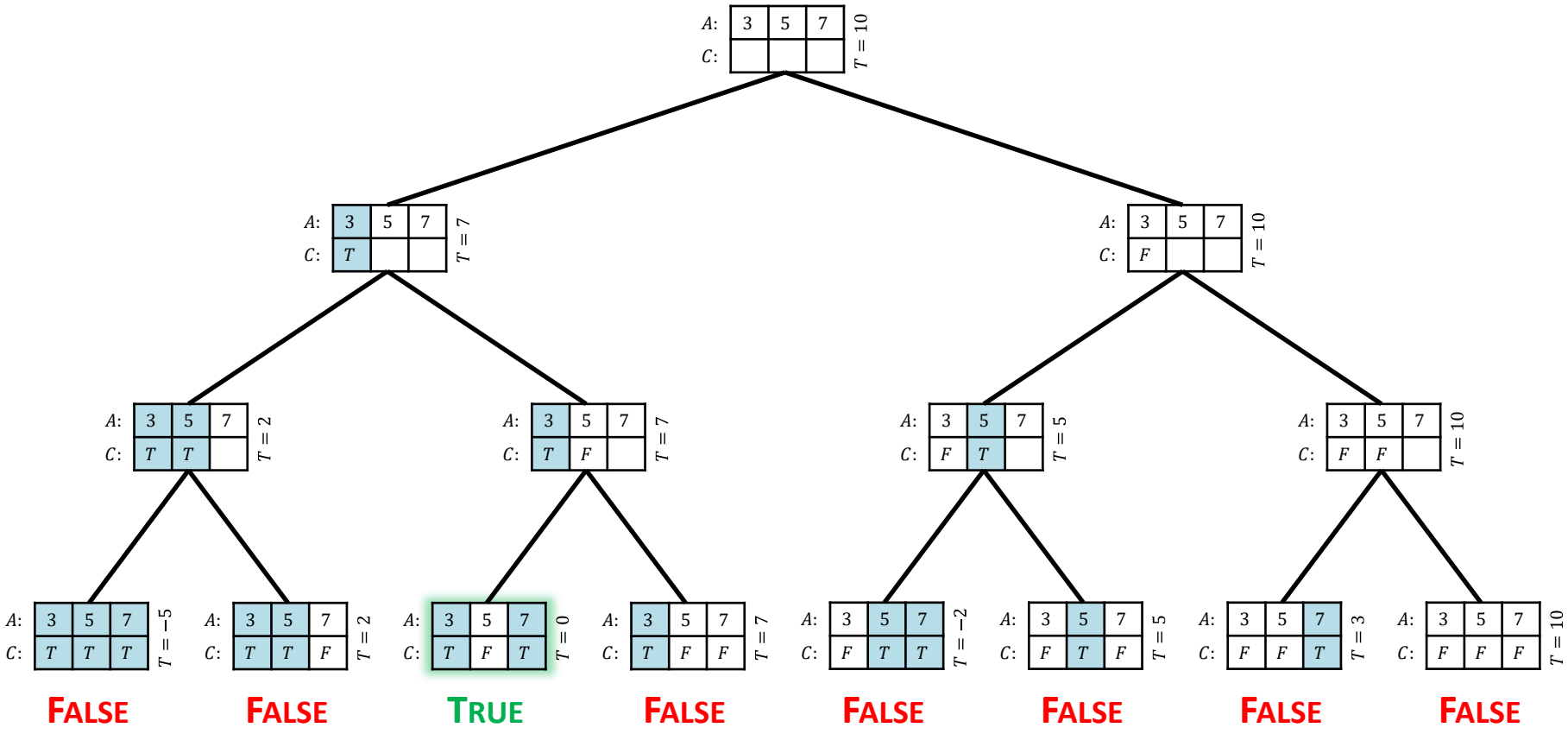
The Subset Sum Problem



The Subset Sum Problem



The Subset Sum Problem



The Subset Sum Problem

Let $T(n)$ be the running time of the algorithm on an input of size n.

Then $$T(n) \leq \begin{cases} \Theta(1), & if\ n = 0, \\ 2T(n-1) + \Theta(1), & otherwise. \end{cases}$$

Solving, $T(n) = O(2^n)$.

String Segmentation

Given a string $S[1..n]$ of length n and an index $i \geq 1$, determine if $S[i..n]$ can be split into meaningful words.

String Segmentation

Given a string $S[1..n]$ of length n and an index $i \geq 1$, determine if $S[i..n]$ can be split into meaningful words.

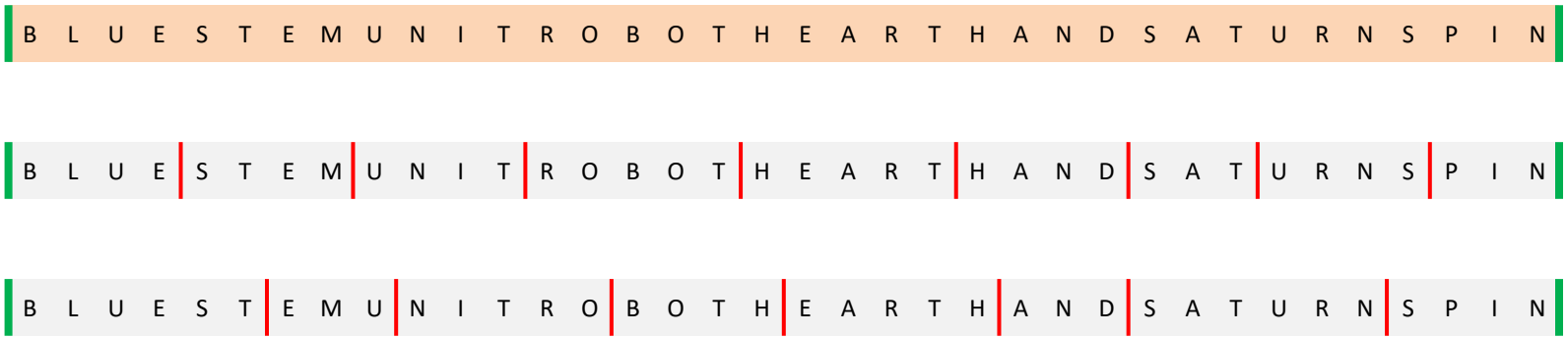
Let $Splittable(i)$ be TRUE iff $S[i..n]$ can be split into meaningful words.

Then

$$Splittable(i) = \begin{cases} \text{TRUE}, & if\ i > n, \\ \bigvee_{j=i}^{n} \left(\text{ISWORD}(i, j) \wedge Splittable(j+1) \right), & otherwise. \end{cases}$$

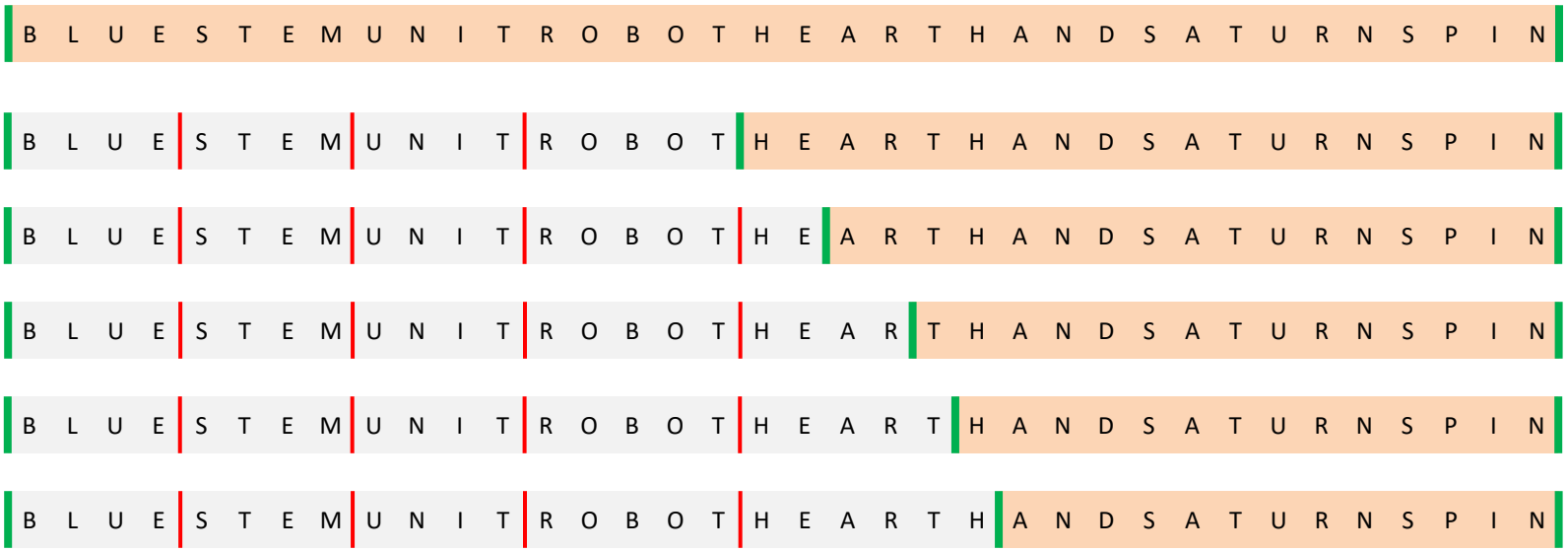
String Segmentation

Given a string $S[1..n]$ of length n and an index $i \geq 1$, determine if $S[i..n]$ can be split into meaningful words.



String Segmentation

Given a string $S[1..n]$ of length n and an index $i \geq 1$, determine if $S[i..n]$ can be split into meaningful words.



String Segmentation

Given a string $S[1..n]$ of length n and an index $i \geq 1$, determine if $S[i..n]$ can be split into meaningful words.

When called as *SPLITTABLE*($S[1:n], n, 1$), the following algorithm returns TRUE if $S[1..n]$ can be split into meaningful words, and FALSE otherwise.

```
SPLITTABLE ( S, n, i )
1.      if i > n then
2.          return TRUE
3.      else
4.          for j ← i to n do
5.              if IS-WORD ( S, i, j ) = TRUE then
6.                  if SPLITTABLE ( S, n, j + 1 ) = TRUE then
7.                      return TRUE
8.          return FALSE
```

String Segmentation

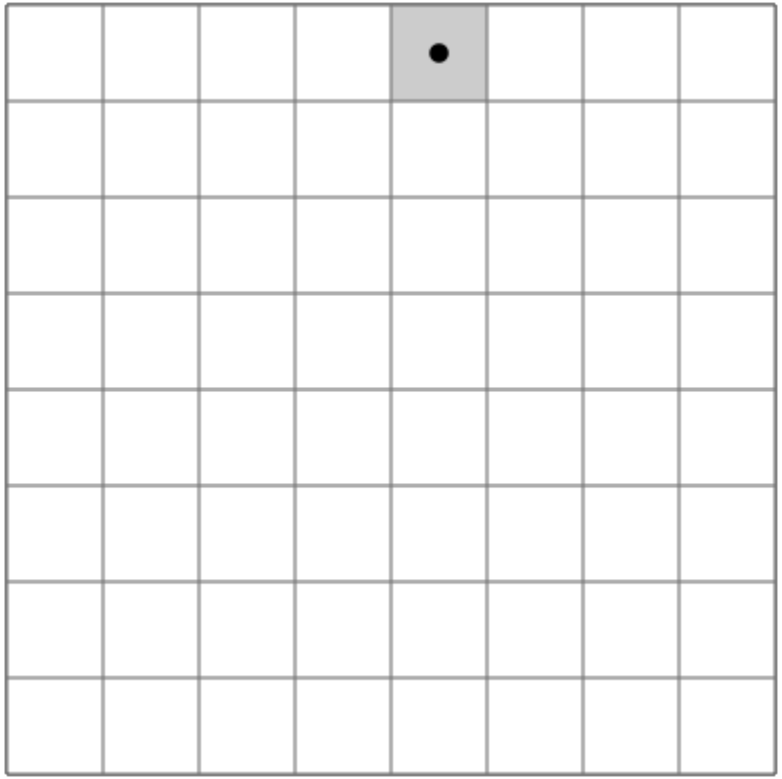
Let $T(n)$ be the running time of the algorithm on an input of size n.

Then

$$T(n) \leq \begin{cases} \Theta(1), & if\ n = 0, \\ \sum_{i=0}^{n-1} T(i) + \mathrm{O}(n), & otherwise. \end{cases}$$

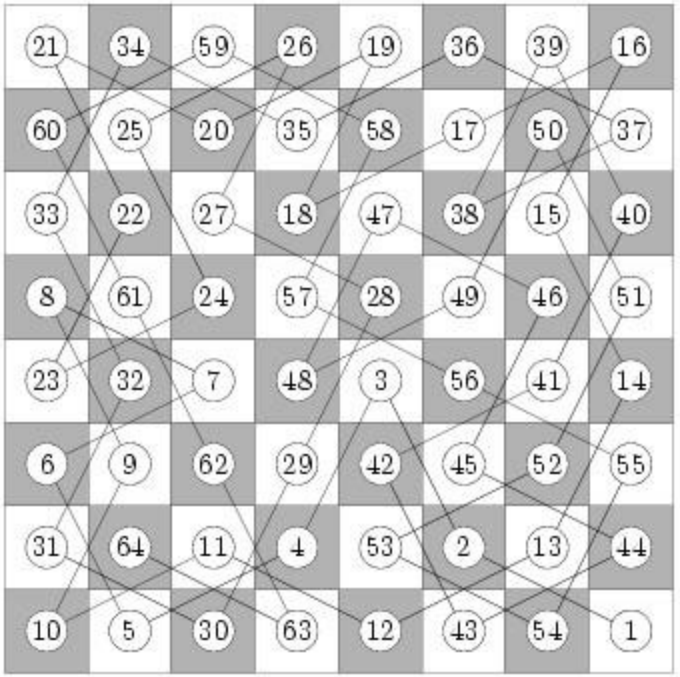
Solving, $T(n) = O(2^n)$.

Knight's Tour



Source: Wikipedia

Knight's Tour



Knight's Tour

Given an $n \times n$ board $B[1..n, 1..n]$ with each cell initialized with a -1, and a location (x, y) with $1 \leq x, y \leq n$, construct a Knight's tour on the board starting at location (x, y).

Knight's Tour

Given an $n \times n$ board $B[1..n, 1..n]$ with each cell initialized with a -1, and a location (x, y) with $1 \leq x, y \leq n$, construct a Knight's tour on the board starting at location (x, y).

If such a tour exists return TRUE, and set $B[x, y] = 1$ and fill each of the remaining board locations with an integer between 1 and n^2 giving the step at which the knight reached that location.

Knight's Tour

Given an $n \times n$ board $B[1..n, 1..n]$ with each cell initialized with a -1, and a location (x, y) with $1 \leq x, y \leq n$, construct a Knight's tour on the board starting at location (x, y).

If such a tour exists return TRUE, and set $B[x, y] = 1$ and fill each of the remaining board locations with an integer between 1 and n^2 giving the step at which the knight reached that location.

If no such tour exists return FALSE.

Knight's Tour

Given an $n \times n$ board $B[1..n, 1..n]$ with each cell initialized with a -1, and a location (x, y) with $1 \leq x, y \leq n$, construct a Knight's tour on the board starting at location (x, y).

If such a tour exists return TRUE, and set $B[x, y] = 1$ and fill each of the remaining board locations with an integer between 1 and n^2 giving the step at which the knight reached that location.

If no such tour exists return FALSE.

For efficiency, use ***Warnsdorff's rule*** to select the next move:

Move the knight to the successor square that itself has the least number of successor squares, where a successor square is defined as any unvisited square to which the knight can next move.

(*https://www.chess.com/blog/kurtgodden/a-tour-of-the-knights-tour*)